

Model10

May 2, 2026

0.0.1 Model 10 - DPO (Direct Preference Optimization) + 7B Model

0.0.2 Replaces PPO entirely - no value network, no rollouts, no NaN risk

0.0.3 Learns from preference pairs: (min correct budget, max wrong budget)

0.0.4 Cell 1: Install (skip if done), restart kernel, then Cell 2

```
[ ]: # Cell 1: Install
import subprocess, sys
subprocess.check_call([sys.executable, '-m', 'pip', 'install',
    'torch', '--index-url', 'https://download.pytorch.org/whl/cu128', '-q'])
subprocess.check_call([sys.executable, '-m', 'pip', 'install',
    'numpy', 'transformers', 'datasets', 'sentence-transformers', 'accelerate',
    '-q'])
import torch
print('torch:', torch.__version__)
print('cuda:', torch.cuda.is_available())
if torch.cuda.is_available(): print('gpu:', torch.cuda.get_device_name(0))
print('DONE - restart kernel then run Cell 2')
```

```
[2]: import os
import re
import json
import copy
import time
import random
import shutil
import hashlib
from dataclasses import dataclass
from typing import Dict, List, Tuple, Optional

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForCausalLM
```

```

from sentence_transformers import SentenceTransformer

# =====
# Config
# =====

@dataclass
class Config:
    # Dataset
    train_subset: int = 300
    test_subset: int = 100

    # Models
    llm_name: str = "Qwen/Qwen2.5-7B-Instruct"
    embed_name: str = "sentence-transformers/all-MiniLM-L6-v2"

    # PARALLELISM: how many questions to process at once on GPU
    # H200 150GB VRAM: 7B model ~15GB, batch=8 safe
    llm_batch_size: int = 8

    # Oracle budgets: only 128-220 where model actually works
    # Removed 96 and 112 because model fails 70-74% there
    oracle_budgets: Tuple[int, ...] = (
        128, 144, 160, 176, 192, 210, 220
    )

    # Supervised warm-start
    warmstart_epochs: int = 10
    warmstart_learning_rate: float = 5e-4

    # PPO specific
    rl_epochs: int = 10
    ppo_epochs: int = 2          # FIX: reduced from 4 to 2 - prevents
    ↪over-updating per rollout

    # DPO specific - replaces all PPO params
    # No value network, no rollouts, no clipping, no NaN risk
    dpo_epochs: int = 20         # epochs over preference pairs
    dpo_batch_size: int = 32     # pairs per batch
    dpo_beta: float = 0.1        # KL penalty strength (0.1 = standard)
    dpo_learning_rate: float = 5e-5 # conservative lr for stable training
    grad_clip_norm: float = 1.0  # standard clip - DPO gradients are
    ↪bounded

    # Reward: rebalanced penalty so 176-token correct gets positive reward
    # At 176 tokens: 1.0 + 0.25 - 0.0005*176 = 1.162 (correct)
    # At 128 tokens: 1.0 + 0.25 - 0.0005*128 = 1.186 (correct)

```

```

# Small gap encourages efficiency without punishing longer budgets
reward_token_penalty: float = 0.0005 # 7B more accurate, moderate penalty
exact_match_bonus: float = 0.25

# Budget range: min raised to 128 (below this model fails reliably)
min_budget: int = 96
max_budget: int = 256

# Answer generation
answer_max_new_tokens: int = 20
do_sample: bool = False
temperature: float = 0.0

# RL warmup exploration in 160-220 where model succeeds
warmup_epochs_rl: int = 2
warmup_mix_rl: float = 0.25
warmup_min_budget_high: int = 160

# Oracle blend not used in DPD (warmstart provides supervised signal)
use_oracle_blend: bool = False
oracle_blend_weight: float = 0.0

# Hardware
seed: int = 42
device: str = "cuda" if torch.cuda.is_available() else "cpu"

# Logging
print_every: int = 20
debug_examples: int = 3
eval_print_every: int = 10

# Cache
cache_dir: str = "./cache_dpo_v17"
clear_cache_on_start: bool = True
oracle_cache_path: str = "oracle_labels_v17.json"

# Saved models
best_warmstart_path: str = "best_warmstart_v17.pt"
best_warmstart_acc_path: str = "best_warmstart_acc_v17.pt"
best_dpo_path: str = "best_dpo_v17.pt"
best_dpo_acc_path: str = "best_dpo_acc_v17.pt"
best_value_path: str = "best_value_v17.pt"
final_policy_path: str = "final_policy_v17.pt"
final_value_path: str = "final_value_v17.pt"

# Baselines
baseline_budgets: Tuple[int, ...] = (96, 128, 160, 192, 224, 256)

```

```

CFG = Config()

# =====
# Utilities
# =====

def set_seed(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

def ensure_dir(path):
    os.makedirs(path, exist_ok=True)

def reset_cache_dir(cache_dir, retries=5, delay=0.5):
    if not os.path.exists(cache_dir):
        os.makedirs(cache_dir, exist_ok=True)
        return
    last_error = None
    for _ in range(retries):
        try:
            if os.path.exists(cache_dir):
                shutil.rmtree(cache_dir, ignore_errors=False)
            if os.path.exists(cache_dir):
                for root, dirs, files in os.walk(cache_dir, topdown=False):
                    for f in files:
                        try: os.remove(os.path.join(root, f))
                        except FileNotFoundError: pass
                    for d in dirs:
                        try: os.rmdir(os.path.join(root, d))
                        except OSError: pass
                if os.path.exists(cache_dir): os.rmdir(cache_dir)
            break
        except OSError as e:
            last_error = e
            time.sleep(delay)
    if os.path.exists(cache_dir):
        raise RuntimeError(f"Failed to clear cache: {cache_dir}") from _
    ↪ last_error
    os.makedirs(cache_dir, exist_ok=True)

```

```

def clamp_budget(x, min_budget, max_budget):
    return int(max(min_budget, min(max_budget, round(x))))

def denormalize_budget(x, min_budget, max_budget):
    return x * (max_budget - min_budget) + min_budget

def budget_to_normalized(budget, min_budget, max_budget):
    return float(budget - min_budget) / float(max_budget - min_budget)

def clean_model_output(text):
    if text is None: return ""
    cleaned = text
    for marker in ["\nHuman:", "\nAssistant:", "\nUser:", "\nSystem:",
                  "Human:", "Assistant:", "User:", "System:",
                  "You are an AI assistant", "<|im_end|>", "<|endoftext|>"]:
        if marker in cleaned:
            cleaned = cleaned.split(marker)[0]
    return cleaned.strip()

# =====
# GSM8K helpers
# =====

def extract_gsm8k_final_answer(answer_text):
    if "####" in answer_text:
        return answer_text.split("####")[-1].strip()
    return answer_text.strip()

def normalize_numeric_string(text):
    if text is None: return ""
    text = text.strip().replace(",", "").replace("$", "").replace("%", "").strip()
    matches = re.findall(r"-?\d+(?:\.\d+)?", text)
    if not matches: return text.lower().strip()
    return matches[-1]

def parse_final_answer(output_text):
    text = clean_model_output(output_text)
    for pattern in [

```

```

        r"FINAL ANSWER\s*:\s*(-?\d+(?:\.\d+)?)",
        r"Final Answer\s*:\s*(-?\d+(?:\.\d+)?)",
        r"Answer\s*:\s*(-?\d+(?:\.\d+)?)",
    ]:
        m = re.search(pattern, text, re.IGNORECASE)
        if m: return normalize_numeric_string(m.group(1))
    lines = [l.strip() for l in text.splitlines() if l.strip()]
    first_line = lines[0] if lines else text.strip()
    m = re.search(r"-?\d+(?:\.\d+)?", first_line)
    if m: return normalize_numeric_string(m.group(0))
    matches = re.findall(r"-?\d+(?:\.\d+)?", text)
    if matches: return normalize_numeric_string(matches[0])
    return normalize_numeric_string(text)

def is_correct(pred, gold):
    return int(pred == gold)

def compute_reward(pred, gold, thinking_tokens, cfg):
    exact = float(is_correct(pred, gold))
    reward = exact - cfg.reward_token_penalty * float(thinking_tokens)
    if exact > 0.5:
        reward += cfg.exact_match_bonus
    return reward, exact

# =====
# Prompts
# =====

def build_thinking_prompt(question):
    return (
        "Solve the following grade-school math problem carefully.\n"
        "Reason step by step. Keep reasoning concise and relevant.\n"
        "Do not start a conversation. Only produce reasoning.\n\n"
        f"Question: {question}\n\nReasoning:"
    )

def build_answer_prompt(question, thinking_text):
    return (
        "Use the reasoning below to produce the final numeric answer.\n"
        "Return only: FINAL ANSWER: <number>\n\n"
        f"Question: {question}\n\nReasoning:\n{thinking_text}\n\nFINAL ANSWER:"
    )

```

```

# =====
# Caching
# =====

def make_cache_key(stage, question, extra, model_name):
    raw = f"{stage}|||{model_name}|||{question}|||{extra}"
    return hashlib.md5(raw.encode()).hexdigest()

def load_cache(cache_dir, key):
    path = os.path.join(cache_dir, f"{key}.json")
    if os.path.exists(path):
        with open(path, "r", encoding="utf-8") as f:
            return json.load(f)
    return None

def save_cache(cache_dir, key, value):
    path = os.path.join(cache_dir, f"{key}.json")
    with open(path, "w", encoding="utf-8") as f:
        json.dump(value, f, ensure_ascii=False, indent=2)

# =====
# LLM Reasoner with BATCHED GPU inference
# This is the main speedup vs Model 5
# Instead of 1 question at a time, processes batch_size questions
# simultaneously on the GPU - uses the H200 properly
# =====

class LLMReasoner:
    def __init__(self, model_name, device, batch_size=16):
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        if self.tokenizer.pad_token is None:
            self.tokenizer.pad_token = self.tokenizer.eos_token

        # LEFT padding is required for decoder-only models in batched mode
        # Without this, the model attends to wrong positions
        self.tokenizer.padding_side = "left"

        self.model = AutoModelForCausalLM.from_pretrained(
            model_name,
            dtype=torch.float16,
            device_map="auto"
        )
        self.model.eval()

```

```

self.device = device
self.batch_size = batch_size

if hasattr(self.model, "generation_config") and self.model.
↳ generation_config:
    self.model.generation_config.do_sample = False
    self.model.generation_config.temperature = None
    self.model.generation_config.top_p = None
    self.model.generation_config.top_k = None

def generate(self, prompt, max_new_tokens, do_sample=False, temperature=0.
↳ 0):
    """Single generation - keeps backward compatibility with cache system.
    ↳ """
    results = self.generate_batch([prompt], max_new_tokens, do_sample,
↳ temperature)
    return results[0]

def generate_batch(self, prompts, max_new_tokens, do_sample=False,
↳ temperature=0.0):
    """
    BATCHED GPU INFERENCE - the key parallelism speedup.

    How it works:
    - Takes a list of prompts
    - Tokenizes them all together with padding
    - GPU processes ALL of them in one forward pass
    - Returns results for all prompts

    Why this is faster:
    - Sequential: GPU runs at 5% utilization (mostly idle waiting)
    - Batched: GPU runs at 70-90% utilization
    - Speedup: 4-8x depending on batch size
    """
    all_results = []

    for i in range(0, len(prompts), self.batch_size):
        batch = prompts[i : i + self.batch_size]

        # Tokenize with padding so all sequences are same length
        inputs = self.tokenizer(
            batch,
            return_tensors="pt",
            padding=True,
            truncation=True,
            max_length=600,
        ).to(self.device)

```



```

input_len = inputs["input_ids"].shape[1]

gen_kwargs = {
    "max_new_tokens": max_new_tokens,
    "do_sample": do_sample,
    "pad_token_id": self.tokenizer.eos_token_id,
    "eos_token_id": self.tokenizer.eos_token_id,
    "attention_mask": inputs["attention_mask"],
}
if do_sample:
    gen_kwargs["temperature"] = temperature

with torch.no_grad():
    # GPU processes entire batch at once
    outputs = self.model.generate(inputs["input_ids"], **gen_kwargs)

for output in outputs:
    gen_ids = output[input_len:]
    text = clean_model_output(
        self.tokenizer.decode(gen_ids, skip_special_tokens=True)
    )
    all_results.append({"text": text, "tokens_used": len(gen_ids)})

return all_results

# =====
# Two-stage example run (single, uses cache)
# =====

def run_two_stage_example(question, gold_answer, thinking_budget, llm, cfg):
    ensure_dir(cfg.cache_dir)

    # Stage 1: Thinking
    think_key = make_cache_key("think", question, f"budget={thinking_budget}",
    ↪cfg.llm_name)
    cached = load_cache(cfg.cache_dir, think_key)
    if cached:
        thinking_text = clean_model_output(cached["text"])
        thinking_tokens = cached["tokens_used"]
    else:
        r = llm.generate(build_thinking_prompt(question), thinking_budget)
        thinking_text = clean_model_output(r["text"])
        thinking_tokens = r["tokens_used"]
        save_cache(cfg.cache_dir, think_key, {"text": thinking_text,
    ↪"tokens_used": thinking_tokens})

```

```

# Stage 2: Answer
answer_key = make_cache_key("answer", question, thinking_text, cfg.llm_name)
cached = load_cache(cfg.cache_dir, answer_key)
if cached:
    answer_text = clean_model_output(cached["text"])
    answer_tokens = cached["tokens_used"]
else:
    r = llm.generate(build_answer_prompt(question, thinking_text), cfg.
↪answer_max_new_tokens)
    answer_text = clean_model_output(r["text"])
    answer_tokens = r["tokens_used"]
    save_cache(cfg.cache_dir, answer_key, {"text": answer_text, ↪
↪"tokens_used": answer_tokens})

pred = parse_final_answer(answer_text)
reward, exact = compute_reward(pred, gold_answer, thinking_tokens, cfg)
return {
    "prediction": pred, "gold": gold_answer, "correct": int(exact),
    "thinking_tokens_used": thinking_tokens, "answer_tokens_used": ↪
↪answer_tokens,
    "reward": reward, "thinking_text": thinking_text, "answer_text": ↪
↪answer_text,
    "thinking_budget": thinking_budget,
}

# =====
# BATCHED two-stage run for Oracle and RL rollouts
# Processes multiple questions at once using GPU parallelism
# =====

def run_batch_two_stage(questions, gold_answers, thinking_budget, llm, cfg):
    """
    Run two-stage inference for a BATCH of questions at the same budget.

    Step 1: Check cache for all questions
    Step 2: Run LLM on uncached questions in ONE batched GPU call
    Step 3: Run answer extraction in ONE batched GPU call
    Step 4: Save new results to cache

    This replaces the sequential for-loop that caused the 4-hour runtime.
    """
    ensure_dir(cfg.cache_dir)
    n = len(questions)

    # --- Stage 1: Thinking ---

```

```

thinking_texts = [None] * n
thinking_tokens_list = [None] * n
uncached_think_idx = []
uncached_think_prompts = []

# Check cache first for all questions
for i, q in enumerate(questions):
    key = make_cache_key("think", q, f"budget={thinking_budget}", cfg.
↳llm_name)
    cached = load_cache(cfg.cache_dir, key)
    if cached:
        thinking_texts[i] = clean_model_output(cached["text"])
        thinking_tokens_list[i] = cached["tokens_used"]
    else:
        uncached_think_idx.append(i)
        uncached_think_prompts.append(build_thinking_prompt(q))

# Batch GPU call for all uncached thinking
if uncached_think_prompts:
    batch_results = llm.generate_batch(uncached_think_prompts,
↳thinking_budget)
    for idx, result in zip(uncached_think_idx, batch_results):
        thinking_texts[idx] = clean_model_output(result["text"])
        thinking_tokens_list[idx] = result["tokens_used"]
        key = make_cache_key("think", questions[idx],
↳f"budget={thinking_budget}", cfg.llm_name)
        save_cache(cfg.cache_dir, key, {
            "text": thinking_texts[idx],
            "tokens_used": thinking_tokens_list[idx]
        })

# --- Stage 2: Answer extraction ---
answer_texts = [None] * n
answer_tokens_list = [None] * n
uncached_ans_idx = []
uncached_ans_prompts = []

for i in range(n):
    key = make_cache_key("answer", questions[i], thinking_texts[i], cfg.
↳llm_name)
    cached = load_cache(cfg.cache_dir, key)
    if cached:
        answer_texts[i] = clean_model_output(cached["text"])
        answer_tokens_list[i] = cached["tokens_used"]
    else:
        uncached_ans_idx.append(i)
        uncached_ans_prompts.append(

```

```

        build_answer_prompt(questions[i], thinking_texts[i])
    )

    # Batch GPU call for all uncached answers
    if uncached_ans_prompts:
        batch_results = llm.generate_batch(uncached_ans_prompts, cfg.
        ↪answer_max_new_tokens)
        for idx, result in zip(uncached_ans_idx, batch_results):
            answer_texts[idx] = clean_model_output(result["text"])
            answer_tokens_list[idx] = result["tokens_used"]
            key = make_cache_key("answer", questions[idx], thinking_texts[idx],
            ↪cfg.llm_name)
            save_cache(cfg.cache_dir, key, {
                "text": answer_texts[idx],
                "tokens_used": answer_tokens_list[idx]
            })

    # --- Compute rewards ---
    results = []
    for i in range(n):
        pred = parse_final_answer(answer_texts[i])
        reward, exact = compute_reward(pred, gold_answers[i],
        ↪thinking_tokens_list[i], cfg)
        results.append({
            "prediction": pred, "gold": gold_answers[i], "correct": int(exact),
            "thinking_tokens_used": thinking_tokens_list[i],
            "answer_tokens_used": answer_tokens_list[i],
            "reward": reward, "thinking_text": thinking_texts[i],
            "answer_text": answer_texts[i], "thinking_budget": thinking_budget,
        })
    return results

# =====
# Featurizer
# =====

class QuestionFeaturizer:
    """
    Rich difficulty-aware featurizer.
    New: 12 difficulty features on top of original 50 handcrafted = 62 total
    ↪handcrafted.
    Total input dim: 384 (SBERT all-MiniLM-L6-v2) + 62 = 446.
    """
    def __init__(self, embed_model_name, device):
        self.embedder = SentenceTransformer(embed_model_name, device=device)

```

```

def handcrafted_features(self, question):
    q = question.lower()
    tokens = q.split()
    nums = re.findall(r"\d+\.?\\d*", q)

    # --- Original 50 features (preserved for backward compat) ---
    orig_keywords = [
        "total", "left", "remaining", "each", "every", "more", "less",
        "after", "before", "twice", "half", "times", "altogether",
        "difference", "sum", "product", "cost", "price", "sold",
        "per", "average", "equal", "share", "then", "gave", "bought",
        "earned", "minutes", "hours", "days", "weeks", "fraction",
        "ratio", "percent", "another", "still", "now", "together"
    ]
    orig_feats = [
        len(tokens),
        len(nums),
        len(q),
        max(1, len(re.findall(r"[.!?]", q))),
        1.0 if any(w in q for w in ["more", "sum", "altogether", "total",
↪ "together"]) else 0.0,
        1.0 if any(w in q for w in ["left", "remaining", "difference",
↪ "less", "still"]) else 0.0,
        1.0 if any(w in q for w in ["each", "every", "times", "product",
↪ "twice"]) else 0.0,
        1.0 if any(w in q for w in ["per", "average", "equally", "half",
↪ "share", "ratio"]) else 0.0,
        1.0 if any(w in q for w in ["then", "after", "before",
↪ "altogether", "remaining", "now"]) else 0.0,
        1.0 if any(w in q for w in ["minutes", "hours", "days", "weeks"])
↪ else 0.0,
        1.0 if "$" in q or any(w in q for w in ["dollar", "dollars",
↪ "cost", "price"]) else 0.0,
        1.0 if any(w in q for w in ["half", "fraction", "percent",
↪ "ratio"]) else 0.0,
    ] + [1.0 if kw in q else 0.0 for kw in orig_keywords]

    # --- NEW: 12 difficulty-detection features ---

    # 1. Operation count - how many arithmetic operations are implied
    op_words = [
        "add", "added", "plus", "more", "total", "sum", "altogether",
↪ "combined",
        "subtract", "subtracted", "minus", "less", "fewer", "remain",
↪ "left", "difference",

```

```

        "multiply", "multiplied", "times", "product", "double", "triple",
↪ "twice",
        "divide", "divided", "split", "share", "each", "per", "half",
↪ "quarter",
        "percent", "percentage", "fraction", "ratio", "rate"
    ]
    op_count = float(sum(1 for w in tokens if w in op_words))

    # 2. Numerical value count
    num_count = float(len(nums))

    # 3. Large number flag (>100 implies non-trivial arithmetic)
    has_large_num = 1.0 if any(
        float(n) > 100 for n in nums
        if re.match(r"^\d+\.? \d*$", n)
    ) else 0.0

    # 4. Sentence count - proxy for reasoning steps required
    sentence_count = float(max(1, len(re.findall(r"[!?]", q)) + 1))

    # 5. Multi-step sequence indicators
    multistep_words = [
        "first", "then", "next", "after that", "finally", "lastly",
        "second", "third", "step", "now", "before", "once"
    ]
    multistep_count = float(sum(1 for w in multistep_words if w in q))

    # 6. Conditional logic (if/when makes questions harder)
    has_conditional = 1.0 if any(w in q for w in [
        "if ", "when ", "unless", "until", "provided"
    ]) else 0.0

    # 7. Comparison operators
    has_comparison = 1.0 if any(w in q for w in [
        "more than", "less than", "greater than", "fewer than",
        "at least", "at most", "how many more", "how many fewer",
        "how much more", "how much less"
    ]) else 0.0

    # 8. Unit conversion (time/distance/weight add steps)
    has_unit_conversion = 1.0 if any(w in q for w in [
        "minute", "hour", "second", "day", "week", "month", "year",
        "inch", "foot", "feet", "yard", "mile", "meter", "kilometer",
        "ounce", "pound", "gram", "kilogram", "liter", "gallon"
    ]) else 0.0

    # 9. Percentage/fraction complexity

```

```

has_pct_fraction = 1.0 if any(w in q for w in [
    "percent", "%", "fraction", "half", "third", "quarter",
    "one-half", "one-third", "two-thirds", "one-quarter"
]) else 0.0

# 10. Entity count (people/objects to track)
entity_indicators = [
    "he", "she", "they", "his", "her", "their",
    "john", "mary", "tom", "jane", "bob", "alice",
    "sam", "mike", "sarah", "david", "lisa", "james"
]
entity_count = float(sum(1 for w in tokens if w in entity_indicators))

# 11. Multiple sub-questions
has_multiple_q = 1.0 if (
    q.count("?") > 1 or "and how" in q or "also how" in q
) else 0.0

# 12. Composite difficulty score (normalized 0-1)
raw_difficulty = (
    min(op_count, 8.0) / 8.0 * 0.35 +
    min(num_count, 6.0) / 6.0 * 0.25 +
    min(sentence_count, 5.0) / 5.0 * 0.20 +
    min(multistep_count, 4.0) / 4.0 * 0.20
)

new_feats = [
    op_count, num_count, has_large_num, sentence_count,
    multistep_count, has_conditional, has_comparison,
    has_unit_conversion, has_pct_fraction, entity_count,
    has_multiple_q, raw_difficulty,
]

return np.array(orig_feats + new_feats, dtype=np.float32)

def encode(self, question):
    emb = self.embedder.encode(
        question, convert_to_numpy=True, normalize_embeddings=True
    ).astype(np.float32)
    return np.concatenate([emb, self.handcrafted_features(question)],
↪axis=0)

def encode_batch(self, questions):
    embs = self.embedder.encode(
        questions, convert_to_numpy=True,
        normalize_embeddings=True, batch_size=64
    ).astype(np.float32)

```

```

        hand = np.stack([self.handcrafted_features(q) for q in questions])
        return np.concatenate([embs, hand], axis=1)

# =====
# Policy + Value Networks
# =====

class ContinuousBudgetPolicy(nn.Module):
    def __init__(self, input_dim, hidden_dim=256):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim), nn.ReLU(),
        )
        self.mean_head = nn.Linear(hidden_dim, 1)
        # log_std = 0.5 gives std ~1.65 for initial exploration
        self.log_std = nn.Parameter(torch.tensor([0.5], dtype=torch.float32))

    def forward(self, x):
        h = self.net(x)
        mean = self.mean_head(h)
        log_std = self.log_std_head(h).clamp(-2, 2)
        if torch.isnan(mean).any() or torch.isnan(log_std).any():
            mean = torch.zeros_like(mean)
            log_std = torch.full_like(log_std, -0.5)
        return mean, log_std

# Model 12: DP0 (Direct Preference Optimization) + 7B Model
# Key fixes over Model 8 (39% at 182 tokens, matched fixed budget 184):
# 1. Oracle uses MINIMUM correct budget per question (not highest reward)
#    Easy q solved at 144 -> label 144; hard q needs 210 -> label 210
#    This teaches real per-question efficiency, not "always use 200"
# 2. Full oracle budget range 128-220 restored (need full range for min_
    ↪ search)
# 3. Token penalty 0.0005->0.001: stronger efficiency incentive
# 4. entropy_coef 0.01->0.015: budget std collapsed to 3.8, need diversity
# 5. rl_epochs 10->12: was still improving at epoch 10
# 6. All Model 8 fixes retained: ppo_epochs=2, value pretrain, PPO clip,
    ↪ batching
# =====

import os
import re
import json
import copy
import time
import random

```



```

import shutil
import hashlib
from dataclasses import dataclass
from typing import Dict, List, Tuple, Optional

import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim

from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForCausalLM
from sentence_transformers import SentenceTransformer

# =====
# Config
# =====

@dataclass
class Config:
    # Dataset
    train_subset: int = 300
    test_subset: int = 100

    # Models
    llm_name: str = "Qwen/Qwen2.5-7B-Instruct"
    embed_name: str = "sentence-transformers/all-MiniLM-L6-v2"

    # PARALLELISM: how many questions to process at once on GPU
    # H200 150GB VRAM: 7B model ~15GB, batch=8 safe
    llm_batch_size: int = 8

    # Oracle budgets: only 128-220 where model actually works
    # Removed 96 and 112 because model fails 70-74% there
    oracle_budgets: Tuple[int, ...] = (
        128, 144, 160, 176, 192, 210, 220
    )

    # Supervised warm-start
    warmstart_epochs: int = 10
    warmstart_learning_rate: float = 5e-4

    # PPO specific
    rl_epochs: int = 10
    ppo_epochs: int = 2          # FIX: reduced from 4 to 2 - prevents
    ↪ over-updating per rollout

```

```

# DPO specific - replaces all PPO params
# No value network, no rollouts, no clipping, no NaN risk
dpo_epochs: int = 20          # epochs over preference pairs
dpo_batch_size: int = 32      # pairs per batch
dpo_beta: float = 0.1         # KL penalty strength (0.1 = standard)
dpo_learning_rate: float = 5e-5 # conservative lr for stable training
grad_clip_norm: float = 1.0   # standard clip - DPO gradients are
↳ bounded

# Reward: rebalanced penalty so 176-token correct gets positive reward
# At 176 tokens:  $1.0 + 0.25 - 0.0005 \cdot 176 = 1.162$  (correct)
# At 128 tokens:  $1.0 + 0.25 - 0.0005 \cdot 128 = 1.186$  (correct)
# Small gap encourages efficiency without punishing longer budgets
reward_token_penalty: float = 0.0005 # 7B more accurate, moderate penalty
exact_match_bonus: float = 0.25

# Budget range: min raised to 128 (below this model fails reliably)
min_budget: int = 96
max_budget: int = 256

# Answer generation
answer_max_new_tokens: int = 20
do_sample: bool = False
temperature: float = 0.0

# RL warmup exploration in 160-220 where model succeeds
warmup_epochs_rl: int = 2
warmup_mix_rl: float = 0.25
warmup_min_budget_high: int = 160

# Oracle blend not used in DPO (warmstart provides supervised signal)
use_oracle_blend: bool = False
oracle_blend_weight: float = 0.0

# Hardware
seed: int = 42
device: str = "cuda" if torch.cuda.is_available() else "cpu"

# Logging
print_every: int = 20
debug_examples: int = 3
eval_print_every: int = 10

# Cache
cache_dir: str = "./cache_dpo_v17"
clear_cache_on_start: bool = True
oracle_cache_path: str = "oracle_labels_v17.json"

```

```

# Saved models
best_warmstart_path: str = "best_warmstart_v17.pt"
best_warmstart_acc_path: str = "best_warmstart_acc_v17.pt"
best_dpo_path: str = "best_dpo_v17.pt"
best_dpo_acc_path: str = "best_dpo_acc_v17.pt"
best_value_path: str = "best_value_v17.pt"
final_policy_path: str = "final_policy_v17.pt"
final_value_path: str = "final_value_v17.pt"

# Baselines
baseline_budgets: Tuple[int, ...] = (96, 128, 160, 192, 224, 256)

CFG = Config()

# =====
# Utilities
# =====

def set_seed(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

def ensure_dir(path):
    os.makedirs(path, exist_ok=True)

def reset_cache_dir(cache_dir, retries=5, delay=0.5):
    if not os.path.exists(cache_dir):
        os.makedirs(cache_dir, exist_ok=True)
        return
    last_error = None
    for _ in range(retries):
        try:
            if os.path.exists(cache_dir):
                shutil.rmtree(cache_dir, ignore_errors=False)
            if os.path.exists(cache_dir):
                for root, dirs, files in os.walk(cache_dir, topdown=False):
                    for f in files:
                        try: os.remove(os.path.join(root, f))
                        except FileNotFoundError: pass

```

```

        for d in dirs:
            try: os.rmdir(os.path.join(root, d))
            except OSError: pass
        if os.path.exists(cache_dir): os.rmdir(cache_dir)
        break
    except OSError as e:
        last_error = e
        time.sleep(delay)
    if os.path.exists(cache_dir):
        raise RuntimeError(f"Failed to clear cache: {cache_dir}") from last_error
    os.makedirs(cache_dir, exist_ok=True)

def clamp_budget(x, min_budget, max_budget):
    return int(max(min_budget, min(max_budget, round(x))))

def denormalize_budget(x, min_budget, max_budget):
    return x * (max_budget - min_budget) + min_budget

def budget_to_normalized(budget, min_budget, max_budget):
    return float(budget - min_budget) / float(max_budget - min_budget)

def clean_model_output(text):
    if text is None: return ""
    cleaned = text
    for marker in ["\nHuman:", "\nAssistant:", "\nUser:", "\nSystem:",
                  "Human:", "Assistant:", "User:", "System:",
                  "You are an AI assistant", "<|im_end|>", "<|endoftext|>"]:
        if marker in cleaned:
            cleaned = cleaned.split(marker)[0]
    return cleaned.strip()

# =====
# GSM8K helpers
# =====

def extract_gsm8k_final_answer(answer_text):
    if "####" in answer_text:
        return answer_text.split("####")[-1].strip()
    return answer_text.strip()

```

```

def normalize_numeric_string(text):
    if text is None: return ""
    text = text.strip().replace(",", "").replace("$", "").replace("%", "").
    ↪strip()
    matches = re.findall(r"-?\d+(?:\.\d+)?", text)
    if not matches: return text.lower().strip()
    return matches[-1]

def parse_final_answer(output_text):
    text = clean_model_output(output_text)
    for pattern in [
        r"FINAL ANSWER\s*:\s*(-?\d+(?:\.\d+)?)",
        r"Final Answer\s*:\s*(-?\d+(?:\.\d+)?)",
        r"Answer\s*:\s*(-?\d+(?:\.\d+)?)",
    ]:
        m = re.search(pattern, text, re.IGNORECASE)
        if m: return normalize_numeric_string(m.group(1))
    lines = [l.strip() for l in text.splitlines() if l.strip()]
    first_line = lines[0] if lines else text.strip()
    m = re.search(r"-?\d+(?:\.\d+)?", first_line)
    if m: return normalize_numeric_string(m.group(0))
    matches = re.findall(r"-?\d+(?:\.\d+)?", text)
    if matches: return normalize_numeric_string(matches[0])
    return normalize_numeric_string(text)

def is_correct(pred, gold):
    return int(pred == gold)

def compute_reward(pred, gold, thinking_tokens, cfg):
    exact = float(is_correct(pred, gold))
    reward = exact - cfg.reward_token_penalty * float(thinking_tokens)
    if exact > 0.5:
        reward += cfg.exact_match_bonus
    return reward, exact

# =====
# Prompts
# =====

def build_thinking_prompt(question):
    return (
        "Solve the following grade-school math problem carefully.\n"
        "Reason step by step. Keep reasoning concise and relevant.\n"
    )

```

```

        "Do not start a conversation. Only produce reasoning.\n\n"
        f"Question: {question}\n\nReasoning:"
    )

def build_answer_prompt(question, thinking_text):
    return (
        "Use the reasoning below to produce the final numeric answer.\n"
        "Return only: FINAL ANSWER: <number>\n\n"
        f"Question: {question}\n\nReasoning:\n{thinking_text}\n\nFINAL ANSWER:"
    )

# =====
# Caching
# =====

def make_cache_key(stage, question, extra, model_name):
    raw = f"{stage}|||{model_name}|||{question}|||{extra}"
    return hashlib.md5(raw.encode()).hexdigest()

def load_cache(cache_dir, key):
    path = os.path.join(cache_dir, f"{key}.json")
    if os.path.exists(path):
        with open(path, "r", encoding="utf-8") as f:
            return json.load(f)
    return None

def save_cache(cache_dir, key, value):
    path = os.path.join(cache_dir, f"{key}.json")
    with open(path, "w", encoding="utf-8") as f:
        json.dump(value, f, ensure_ascii=False, indent=2)

# =====
# LLM Reasoner with BATCHED GPU inference
# This is the main speedup vs Model 5
# Instead of 1 question at a time, processes batch_size questions
# simultaneously on the GPU - uses the H200 properly
# =====

class LLMReasoner:
    def __init__(self, model_name, device, batch_size=16):
        self.tokenizer = AutoTokenizer.from_pretrained(model_name)
        if self.tokenizer.pad_token is None:

```

```

        self.tokenizer.pad_token = self.tokenizer.eos_token

        # LEFT padding is required for decoder-only models in batched mode
        # Without this, the model attends to wrong positions
        self.tokenizer.padding_side = "left"

        self.model = AutoModelForCausalLM.from_pretrained(
            model_name,
            dtype=torch.float16,
            device_map="auto"
        )
        self.model.eval()
        self.device = device
        self.batch_size = batch_size

        if hasattr(self.model, "generation_config") and self.model.
↪generation_config:
            self.model.generation_config.do_sample = False
            self.model.generation_config.temperature = None
            self.model.generation_config.top_p = None
            self.model.generation_config.top_k = None

        def generate(self, prompt, max_new_tokens, do_sample=False, temperature=0.
↪0):
            """Single generation - keeps backward compatibility with cache system.
            ↪"""
            results = self.generate_batch([prompt], max_new_tokens, do_sample,
↪temperature)
            return results[0]

        def generate_batch(self, prompts, max_new_tokens, do_sample=False,
↪temperature=0.0):
            """
            BATCHED GPU INFERENCE - the key parallelism speedup.

            How it works:
            - Takes a list of prompts
            - Tokenizes them all together with padding
            - GPU processes ALL of them in one forward pass
            - Returns results for all prompts

            Why this is faster:
            - Sequential: GPU runs at 5% utilization (mostly idle waiting)
            - Batched: GPU runs at 70-90% utilization
            - Speedup: 4-8x depending on batch size
            """
            all_results = []

```

```

for i in range(0, len(prompts), self.batch_size):
    batch = prompts[i : i + self.batch_size]

    # Tokenize with padding so all sequences are same length
    inputs = self.tokenizer(
        batch,
        return_tensors="pt",
        padding=True,
        truncation=True,
        max_length=600,
    ).to(self.device)

    input_len = inputs["input_ids"].shape[1]

    gen_kwargs = {
        "max_new_tokens": max_new_tokens,
        "do_sample": do_sample,
        "pad_token_id": self.tokenizer.eos_token_id,
        "eos_token_id": self.tokenizer.eos_token_id,
        "attention_mask": inputs["attention_mask"],
    }
    if do_sample:
        gen_kwargs["temperature"] = temperature

    with torch.no_grad():
        # GPU processes entire batch at once
        outputs = self.model.generate(inputs["input_ids"], **gen_kwargs)

    for output in outputs:
        gen_ids = output[input_len:]
        text = clean_model_output(
            self.tokenizer.decode(gen_ids, skip_special_tokens=True)
        )
        all_results.append({"text": text, "tokens_used": len(gen_ids)})

    return all_results

# =====
# Two-stage example run (single, uses cache)
# =====

def run_two_stage_example(question, gold_answer, thinking_budget, llm, cfg):
    ensure_dir(cfg.cache_dir)

    # Stage 1: Thinking

```



```

    think_key = make_cache_key("think", question, f"budget={thinking_budget}",  

    ↪cfg.llm_name)  

    cached = load_cache(cfg.cache_dir, think_key)  

    if cached:  

        thinking_text = clean_model_output(cached["text"])  

        thinking_tokens = cached["tokens_used"]  

    else:  

        r = llm.generate(build_thinking_prompt(question), thinking_budget)  

        thinking_text = clean_model_output(r["text"])  

        thinking_tokens = r["tokens_used"]  

        save_cache(cfg.cache_dir, think_key, {"text": thinking_text,  

    ↪"tokens_used": thinking_tokens})  

  

    # Stage 2: Answer  

    answer_key = make_cache_key("answer", question, thinking_text, cfg.llm_name)  

    cached = load_cache(cfg.cache_dir, answer_key)  

    if cached:  

        answer_text = clean_model_output(cached["text"])  

        answer_tokens = cached["tokens_used"]  

    else:  

        r = llm.generate(build_answer_prompt(question, thinking_text), cfg.  

    ↪answer_max_new_tokens)  

        answer_text = clean_model_output(r["text"])  

        answer_tokens = r["tokens_used"]  

        save_cache(cfg.cache_dir, answer_key, {"text": answer_text,  

    ↪"tokens_used": answer_tokens})  

  

    pred = parse_final_answer(answer_text)  

    reward, exact = compute_reward(pred, gold_answer, thinking_tokens, cfg)  

    return {  

        "prediction": pred, "gold": gold_answer, "correct": int(exact),  

        "thinking_tokens_used": thinking_tokens, "answer_tokens_used":  

    ↪answer_tokens,  

        "reward": reward, "thinking_text": thinking_text, "answer_text":  

    ↪answer_text,  

        "thinking_budget": thinking_budget,  

    }  

  

    # =====  

    # BATCHED two-stage run for Oracle and RL rollouts  

    # Processes multiple questions at once using GPU parallelism  

    # =====  

  

    def run_batch_two_stage(questions, gold_answers, thinking_budget, llm, cfg):  

        """

```

Run two-stage inference for a BATCH of questions at the same budget.

Step 1: Check cache for all questions

Step 2: Run LLM on uncached questions in ONE batched GPU call

Step 3: Run answer extraction in ONE batched GPU call

Step 4: Save new results to cache

This replaces the sequential for-loop that caused the 4-hour runtime.
"""

```
ensure_dir(cfg.cache_dir)
n = len(questions)

# --- Stage 1: Thinking ---
thinking_texts = [None] * n
thinking_tokens_list = [None] * n
uncached_think_idx = []
uncached_think_prompts = []

# Check cache first for all questions
for i, q in enumerate(questions):
    key = make_cache_key("think", q, f"budget={thinking_budget}", cfg.
↳llm_name)
    cached = load_cache(cfg.cache_dir, key)
    if cached:
        thinking_texts[i] = clean_model_output(cached["text"])
        thinking_tokens_list[i] = cached["tokens_used"]
    else:
        uncached_think_idx.append(i)
        uncached_think_prompts.append(build_thinking_prompt(q))

# Batch GPU call for all uncached thinking
if uncached_think_prompts:
    batch_results = llm.generate_batch(uncached_think_prompts,
↳thinking_budget)
    for idx, result in zip(uncached_think_idx, batch_results):
        thinking_texts[idx] = clean_model_output(result["text"])
        thinking_tokens_list[idx] = result["tokens_used"]
        key = make_cache_key("think", questions[idx],
↳f"budget={thinking_budget}", cfg.llm_name)
        save_cache(cfg.cache_dir, key, {
            "text": thinking_texts[idx],
            "tokens_used": thinking_tokens_list[idx]
        })

# --- Stage 2: Answer extraction ---
answer_texts = [None] * n
answer_tokens_list = [None] * n
```

```

uncached_ans_idx = []
uncached_ans_prompts = []

for i in range(n):
    key = make_cache_key("answer", questions[i], thinking_texts[i], cfg.
↳llm_name)
    cached = load_cache(cfg.cache_dir, key)
    if cached:
        answer_texts[i] = clean_model_output(cached["text"])
        answer_tokens_list[i] = cached["tokens_used"]
    else:
        uncached_ans_idx.append(i)
        uncached_ans_prompts.append(
            build_answer_prompt(questions[i], thinking_texts[i])
        )

# Batch GPU call for all uncached answers
if uncached_ans_prompts:
    batch_results = llm.generate_batch(uncached_ans_prompts, cfg.
↳answer_max_new_tokens)
    for idx, result in zip(uncached_ans_idx, batch_results):
        answer_texts[idx] = clean_model_output(result["text"])
        answer_tokens_list[idx] = result["tokens_used"]
        key = make_cache_key("answer", questions[idx], thinking_texts[idx],
↳cfg.llm_name)
        save_cache(cfg.cache_dir, key, {
            "text": answer_texts[idx],
            "tokens_used": answer_tokens_list[idx]
        })

# --- Compute rewards ---
results = []
for i in range(n):
    pred = parse_final_answer(answer_texts[i])
    reward, exact = compute_reward(pred, gold_answers[i],
↳thinking_tokens_list[i], cfg)
    results.append({
        "prediction": pred, "gold": gold_answers[i], "correct": int(exact),
        "thinking_tokens_used": thinking_tokens_list[i],
        "answer_tokens_used": answer_tokens_list[i],
        "reward": reward, "thinking_text": thinking_texts[i],
        "answer_text": answer_texts[i], "thinking_budget": thinking_budget,
    })
return results

# =====

```

```

# Featurizer
# =====

class QuestionFeaturizer:
    """
    Rich difficulty-aware featurizer.
    New: 12 difficulty features on top of original 50 handcrafted = 62 total,
    ↪ handcrafted.
    Total input dim: 384 (SBERT all-MiniLM-L6-v2) + 62 = 446.
    """
    def __init__(self, embed_model_name, device):
        self.embedder = SentenceTransformer(embed_model_name, device=device)

    def handcrafted_features(self, question):
        q = question.lower()
        tokens = q.split()
        nums = re.findall(r"\d+\.?\d*", q)

        # --- Original 50 features (preserved for backward compat) ---
        orig_keywords = [
            "total", "left", "remaining", "each", "every", "more", "less",
            "after", "before", "twice", "half", "times", "altogether",
            "difference", "sum", "product", "cost", "price", "sold",
            "per", "average", "equal", "share", "then", "gave", "bought",
            "earned", "minutes", "hours", "days", "weeks", "fraction",
            "ratio", "percent", "another", "still", "now", "together"
        ]
        orig_feats = [
            len(tokens),
            len(nums),
            len(q),
            max(1, len(re.findall(r"[.!?]", q))),
            1.0 if any(w in q for w in ["more", "sum", "altogether", "total",
            ↪ "together"]) else 0.0,
            1.0 if any(w in q for w in ["left", "remaining", "difference",
            ↪ "less", "still"]) else 0.0,
            1.0 if any(w in q for w in ["each", "every", "times", "product",
            ↪ "twice"]) else 0.0,
            1.0 if any(w in q for w in ["per", "average", "equally", "half",
            ↪ "share", "ratio"]) else 0.0,
            1.0 if any(w in q for w in ["then", "after", "before",
            ↪ "altogether", "remaining", "now"]) else 0.0,
            1.0 if any(w in q for w in ["minutes", "hours", "days", "weeks"])
            ↪ else 0.0,
            1.0 if "$" in q or any(w in q for w in ["dollar", "dollars",
            ↪ "cost", "price"]) else 0.0,

```

```

        1.0 if any(w in q for w in ["half", "fraction", "percent",
↪"ratio"])) else 0.0,
    ] + [1.0 if kw in q else 0.0 for kw in orig_keywords]

    # --- NEW: 12 difficulty-detection features ---

    # 1. Operation count - how many arithmetic operations are implied
    op_words = [
        "add", "added", "plus", "more", "total", "sum", "altogether",
↪"combined",
        "subtract", "subtracted", "minus", "less", "fewer", "remain",
↪"left", "difference",
        "multiply", "multiplied", "times", "product", "double", "triple",
↪"twice",
        "divide", "divided", "split", "share", "each", "per", "half",
↪"quarter",
        "percent", "percentage", "fraction", "ratio", "rate"
    ]
    op_count = float(sum(1 for w in tokens if w in op_words))

    # 2. Numerical value count
    num_count = float(len(nums))

    # 3. Large number flag (>100 implies non-trivial arithmetic)
    has_large_num = 1.0 if any(
        float(n) > 100 for n in nums
        if re.match(r"^\d+\.?d*$", n)
    ) else 0.0

    # 4. Sentence count - proxy for reasoning steps required
    sentence_count = float(max(1, len(re.findall(r"[!?]", q)) + 1))

    # 5. Multi-step sequence indicators
    multistep_words = [
        "first", "then", "next", "after that", "finally", "lastly",
        "second", "third", "step", "now", "before", "once"
    ]
    multistep_count = float(sum(1 for w in multistep_words if w in q))

    # 6. Conditional logic (if/when makes questions harder)
    has_conditional = 1.0 if any(w in q for w in [
        "if ", "when ", "unless", "until", "provided"
    ]) else 0.0

    # 7. Comparison operators
    has_comparison = 1.0 if any(w in q for w in [
        "more than", "less than", "greater than", "fewer than",

```

```

        "at least", "at most", "how many more", "how many fewer",
        "how much more", "how much less"
    ]) else 0.0

# 8. Unit conversion (time/distance/weight add steps)
has_unit_conversion = 1.0 if any(w in q for w in [
    "minute", "hour", "second", "day", "week", "month", "year",
    "inch", "foot", "feet", "yard", "mile", "meter", "kilometer",
    "ounce", "pound", "gram", "kilogram", "liter", "gallon"
]) else 0.0

# 9. Percentage/fraction complexity
has_pct_fraction = 1.0 if any(w in q for w in [
    "percent", "%", "fraction", "half", "third", "quarter",
    "one-half", "one-third", "two-thirds", "one-quarter"
]) else 0.0

# 10. Entity count (people/objects to track)
entity_indicators = [
    "he", "she", "they", "his", "her", "their",
    "john", "mary", "tom", "jane", "bob", "alice",
    "sam", "mike", "sarah", "david", "lisa", "james"
]
entity_count = float(sum(1 for w in tokens if w in entity_indicators))

# 11. Multiple sub-questions
has_multiple_q = 1.0 if (
    q.count("?") > 1 or "and how" in q or "also how" in q
) else 0.0

# 12. Composite difficulty score (normalized 0-1)
raw_difficulty = (
    min(op_count, 8.0) / 8.0 * 0.35 +
    min(num_count, 6.0) / 6.0 * 0.25 +
    min(sentence_count, 5.0) / 5.0 * 0.20 +
    min(multistep_count, 4.0) / 4.0 * 0.20
)

new_feats = [
    op_count, num_count, has_large_num, sentence_count,
    multistep_count, has_conditional, has_comparison,
    has_unit_conversion, has_pct_fraction, entity_count,
    has_multiple_q, raw_difficulty,
]

return np.array(orig_feats + new_feats, dtype=np.float32)

```

```

def encode(self, question):
    emb = self.embedder.encode(
        question, convert_to_numpy=True, normalize_embeddings=True
    ).astype(np.float32)
    return np.concatenate([emb, self.handcrafted_features(question)],
↪axis=0)

def encode_batch(self, questions):
    embs = self.embedder.encode(
        questions, convert_to_numpy=True,
        normalize_embeddings=True, batch_size=64
    ).astype(np.float32)
    hand = np.stack([self.handcrafted_features(q) for q in questions])
    return np.concatenate([embs, hand], axis=1)

# =====
# Policy + Value Networks
# =====

class ContinuousBudgetPolicy(nn.Module):
    def __init__(self, input_dim, hidden_dim=256):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim), nn.ReLU(),
        )
        self.mean_head = nn.Linear(hidden_dim, 1)
        # log_std = 0.5 gives std ~1.65 for initial exploration
        self.log_std = nn.Parameter(torch.tensor([0.5], dtype=torch.float32))

    def forward(self, x):
        h = self.net(x)
        mean = self.mean_head(h)
        return mean, self.log_std.expand_as(mean)

    def sample_budget(self, x):
        mean, log_std = self.forward(x)
        dist = torch.distributions.Normal(mean, torch.exp(log_std))
        raw = dist.rsample()
        normalized = torch.sigmoid(raw)
        log_prob = dist.log_prob(raw).sum(dim=-1)
        entropy = dist.entropy().sum(dim=-1)
        return normalized, log_prob, entropy

    def get_log_prob(self, x, normalized_budgets):
        """

```

```

Get log probability for given normalized budgets.
Used in PPO to compute ratio between new and old policy.
"""

    mean, log_std = self.forward(x)
    dist = torch.distributions.Normal(mean, torch.exp(log_std))
    # Inverse sigmoid to get raw values
    raw = torch.logit(normalized_budgets.unsqueeze(-1).clamp(1e-6, 1 - 1e-6))
    log_prob = dist.log_prob(raw).sum(dim=-1)
    entropy = dist.entropy().sum(dim=-1)
    return log_prob, entropy

def deterministic_budget(self, x):
    mean, _ = self.forward(x)
    return torch.sigmoid(mean)

class ValueNetwork(nn.Module):
    def __init__(self, input_dim, hidden_dim=256):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim), nn.ReLU(),
            nn.Linear(hidden_dim, 1),
        )

    def forward(self, x):
        return self.net(x).squeeze(-1)

# =====
# Evaluation
# =====

def evaluate_fixed_budget(data, llm, cfg, fixed_budget):
    corrects, tokens, rewards = [], [], []
    questions = [item["question"] for item in data]
    gold_answers = [
        normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))
        for item in data
    ]
    print(f"\nEvaluating fixed budget = {fixed_budget}...")

    # Process in batches
    for i in range(0, len(questions), cfg.llm_batch_size):
        batch_q = questions[i : i + cfg.llm_batch_size]
        batch_g = gold_answers[i : i + cfg.llm_batch_size]

```



```

        results = run_batch_two_stage(batch_q, batch_g, fixed_budget, llm, cfg)
        for r in results:
            corrects.append(r["correct"])
            tokens.append(r["thinking_tokens_used"])
            rewards.append(r["reward"])
        done = min(i + cfg.llm_batch_size, len(questions))
        if done % cfg.eval_print_every == 0 or done == len(questions):
            print(f" Fixed {fixed_budget} | {done}/{len(questions)} | "
                  f"Acc={np.mean(corrects):.4f} | AvgReward={np.mean(rewards):.4f}")

    return {
        "accuracy": float(np.mean(corrects)),
        "avg_thinking_tokens": float(np.mean(tokens)),
        "avg_reward": float(np.mean(rewards)),
        "budget": fixed_budget
    }

def evaluate_policy_deterministic(data, featurizer, policy, llm, cfg):
    policy.eval()
    corrects, tokens, rewards, budgets = [], [], [], []

    questions = [item["question"] for item in data]
    gold_answers = [
        normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))
        for item in data
    ]

    # Encode all features at once
    all_feats = featurizer.encode_batch(questions)

    with torch.no_grad():
        feats_tensor = torch.tensor(all_feats, dtype=torch.float32, device=cfg.device)
        norm_budgets = policy.deterministic_budget(feats_tensor).squeeze(-1)
        budget_list = [
            clamp_budget(
                denormalize_budget(nb.item(), cfg.min_budget, cfg.max_budget),
                cfg.min_budget, cfg.max_budget
            )
            for nb in norm_budgets
        ]

    # Group questions by budget for batched inference
    from collections import defaultdict
    budget_groups = defaultdict(list)

```

```

for i, b in enumerate(budget_list):
    budget_groups[b].append(i)

results_by_idx = {}
for budget, indices in budget_groups.items():
    batch_q = [questions[i] for i in indices]
    batch_g = [gold_answers[i] for i in indices]
    batch_results = run_batch_two_stage(batch_q, batch_g, budget, llm, cfg)
    for i, r in zip(indices, batch_results):
        results_by_idx[i] = r

for i in range(len(data)):
    r = results_by_idx[i]
    corrects.append(r["correct"])
    tokens.append(r["thinking_tokens_used"])
    rewards.append(r["reward"])
    budgets.append(budget_list[i])

if corrects:
    print(f" Det eval complete | Acc={np.mean(corrects):.4f} | "
          f"AvgBudget={np.mean(budgets):.1f} | StdBudget={np.std(budgets):.1f} | "
          f"Range=[{np.min(budgets):.0f}, {np.max(budgets):.0f}]")

return {
    "accuracy": float(np.mean(corrects)),
    "avg_thinking_tokens": float(np.mean(tokens)),
    "avg_reward": float(np.mean(rewards)),
    "avg_budget": float(np.mean(budgets)),
    "min_budget": float(np.min(budgets)),
    "max_budget": float(np.max(budgets)),
    "std_budget": float(np.std(budgets)),
}

# =====
# Oracle builder using BATCHED inference
# Model 5 took ~2.5 hours for this step alone
# With batching: processes 16 questions per budget in one GPU call
# Expected speedup: 3-5x (from ~2.5 hours to ~30-45 min)
# =====

def build_oracle_budget_labels(train_data, llm, cfg):
    if os.path.exists(cfg.oracle_cache_path):
        print(f"\nLoading oracle labels from {cfg.oracle_cache_path}...")
        with open(cfg.oracle_cache_path, "r", encoding="utf-8") as f:
            return json.load(f)

```

```

print("\nBuilding oracle labels with BATCHED GPU inference...")
questions = [item["question"] for item in train_data]
gold_answers = [
    normalize_numeric_string(extract_gsm8k_final_answer(item["answer"]))
    for item in train_data
]
n = len(questions)

# Initialize records
oracle_records = [
    {"question": q, "gold_answer": g, "budget_rewards": {}, "any_correct": False}
    for q, g in zip(questions, gold_answers)
]

# For each budget, process ALL 300 questions in batches
# This is much faster than the nested loop in Model 5
for budget in cfg.oracle_budgets:
    print(f" Trying budget={budget} for all {n} questions...")
    t0 = time.time()

    for i in range(0, n, cfg.llm_batch_size):
        batch_q = questions[i : i + cfg.llm_batch_size]
        batch_g = gold_answers[i : i + cfg.llm_batch_size]
        batch_results = run_batch_two_stage(batch_q, batch_g, budget, llm, cfg)

        for j, result in enumerate(batch_results):
            idx = i + j
            oracle_records[idx]["budget_rewards"][str(budget)] = {
                "reward": float(result["reward"]),
                "correct": int(result["correct"]),
                "thinking_tokens_used": int(result["thinking_tokens_used"]),
                "prediction": result["prediction"],
            }
            if result["correct"] == 1:
                oracle_records[idx]["any_correct"] = True

    elapsed = time.time() - t0
    correct_at_budget = sum(
        1 for r in oracle_records
        if r["budget_rewards"].get(str(budget), {}).get("correct", 0) == 1
    )
    print(f" Budget {budget} done in {elapsed:.1f}s | "
          f"Correct: {correct_at_budget}/{n} ({100*correct_at_budget/n:.  

1f}%)")

```

```

# Assign best budget per question
# FIX from Model 5: use HIGHEST REWARD budget, not minimum correct
# Model 5 assigned 57% of questions to budget=96 which skewed everything
for record in oracle_records:
    br = record["budget_rewards"]
    correct_budgets = [int(b) for b, v in br.items() if v["correct"] == 1]
    if correct_budgets:
        best_budget = min(correct_budgets)
    else:
        best_budget = int(max(br.keys(), key=lambda b: br[b]["reward"]))
    record["best_budget"] = best_budget
    record["best_reward"] = float(br[str(best_budget)]["reward"])

# Print distribution to verify it looks reasonable
best_budgets = [r["best_budget"] for r in oracle_records]
solvable = [r["any_correct"] for r in oracle_records]
print(f"\nOracle stats: mean={np.mean(best_budgets):.1f}, "
      f"std={np.std(best_budgets):.1f}, "
      f"solvable={np.mean(solvable):.3f}")
for b in sorted(set(best_budgets)):
    count = sum(1 for x in best_budgets if x == b)
    print(f"    budget={b}: {count} questions ({100*count/len(best_budgets):.1f}%)")

with open(cfg.oracle_cache_path, "w", encoding="utf-8") as f:
    json.dump(oracle_records, f, ensure_ascii=False, indent=2)
print(f"Saved oracle labels to {cfg.oracle_cache_path}")
return oracle_records

# =====
# Supervised warm-start
# =====

def warmstart_policy(oracle_records, test_data, featurizer, policy, llm, cfg):
    print("\nStarting supervised warm-start...")
    optimizer = optim.Adam(policy.parameters(), lr=cfg.warmstart_learning_rate)
    mse_loss = nn.MSELoss(reduction="none")

    all_budgets = np.array([r["best_budget"] for r in oracle_records], dtype=np.
↪float32)
    budget_mean = float(all_budgets.mean())
    budget_std = float(all_budgets.std()) + 1e-8

    best_reward = -1e9
    best_accuracy = -1.0

```

```

for epoch in range(cfg.warmstart_epochs):
    policy.train()
    random.shuffle(oracle_records)
    losses, pred_budgets, target_budgets = [], [], []

    for step_idx, record in enumerate(oracle_records):
        target_budget = int(record["best_budget"])
        x = torch.tensor(
            featurizer.encode(record["question"]),
            dtype=torch.float32, device=cfg.device
        ).unsqueeze(0)

        pred_norm = policy.deterministic_budget(x)
        target_norm = torch.tensor(
            [[budget_to_normalized(target_budget, cfg.min_budget, cfg.
↪max_budget)]],
            dtype=torch.float32, device=cfg.device
        )

        # Weight by reward quality + how much this question's budget
↪differs from mean
        reward_weight = max(0.1, float(record["best_reward"]))
        deviation_weight = 1.0 + abs(target_budget - budget_mean) /
↪budget_std
        loss = reward_weight * deviation_weight * mse_loss(pred_norm,
↪target_norm).mean()

        optimizer.zero_grad()
        loss.backward()
        torch.nn.utils.clip_grad_norm_(policy.parameters(), cfg.
↪grad_clip_norm)
        # NaN guard: skip update if gradients exploded
        has_nan_grad = any(
            p.grad is not None and torch.isnan(p.grad).any()
            for p in policy.parameters()
        )
        if not has_nan_grad:
            optimizer.step()
        else:
            print(" WARNING: NaN gradient detected, skipping update")
            optimizer.zero_grad()

        losses.append(float(loss.item()))
        pred_budget = clamp_budget(
            denormalize_budget(pred_norm.item(), cfg.min_budget, cfg.
↪max_budget),

```

```

        cfg.min_budget, cfg.max_budget
    )
    pred_budgets.append(pred_budget)
    target_budgets.append(target_budget)

    if (step_idx + 1) % cfg.print_every == 0 or (step_idx + 1) == len
    ↪ len(oracle_records):
        mae = np.mean(np.abs(np.array(pred_budgets) - np.
    ↪ array(target_budgets)))
        print(f"Warmstart {epoch+1}/{cfg.warmstart_epochs} "
              f"Step {step_idx+1}/{len(oracle_records)} | "
              f"Loss={np.mean(losses):.4f} | "
              f"PredAvg={np.mean(pred_budgets):.1f} | "
              f"TargetAvg={np.mean(target_budgets):.1f} | MAE={mae:.
    ↪ 2f}")

        print(f"Warmstart epoch {epoch+1} done | Loss={np.mean(losses):.4f}")
        eval_det = evaluate_policy_deterministic(test_data, featurizer, policy, llm,
    ↪ cfg)

        if eval_det["avg_reward"] > best_reward:
            best_reward = eval_det["avg_reward"]
            torch.save(policy.state_dict(), cfg.best_warmstart_path)
            print(f" Saved best warmstart by reward = {best_reward:.4f}")

        if eval_det["accuracy"] > best_accuracy:
            best_accuracy = eval_det["accuracy"]
            torch.save(policy.state_dict(), cfg.best_warmstart_acc_path)
            print(f" Saved best warmstart by accuracy = {best_accuracy:.4f}")

# =====
# PPO Training
# This is the core fix over Model 5's basic policy gradient
#
# How PPO works here:
# 1. ROLLOUT: collect 300 (question, budget, reward) experiences
#             using current policy - all done with batched GPU calls
# 2. UPDATE: run 4 epochs of mini-batch updates using the PPO clip
#             The clip prevents any single update from being too large
# 3. REPEAT: for each RL epoch
#
# Why clip stops the oscillation:
# - Old code: loss = -(log_prob * advantage) → unbounded update size
# - PPO:      ratio = exp(new_log_prob - old_log_prob)
#             loss = -min(ratio*A, clip(ratio, 0.8, 1.2)*A)
#             If ratio > 1.2 or < 0.8, gradient is zeroed out

```

```

# Policy cannot change by more than 20% per update
# =====

def build_dpo_pairs(oracle_records, cfg):
    """
    Build preference pairs from oracle records for DPO training.

    For each question:
        preferred = budget where model gets correct answer (lowest token cost)
        rejected  = budget where model gets wrong answer (highest token cost)

    This teaches the policy: "use fewer tokens when you can get it right,
    use more tokens when you need them."
    """
    pairs = []
    for record in oracle_records:
        br = record["budget_rewards"]
        q = record["question"]

        correct_budgets = sorted([int(b) for b, v in br.items() if v["correct"]])
        wrong_budgets = sorted([int(b) for b, v in br.items() if v["correct"]])

        if not correct_budgets or not wrong_budgets:
            continue # need at least one of each

        # Preferred: min budget that gets it right (efficient correct answer)
        preferred_budget = correct_budgets[0]
        # Rejected: max budget that still gets it wrong (wasteful wrong answer)
        rejected_budget = wrong_budgets[-1]

        # Only keep pairs where preferred < rejected (clear efficiency signal)
        # If preferred > rejected, the signal is ambiguous
        if preferred_budget >= rejected_budget:
            continue

        preferred_norm = budget_to_normalized(preferred_budget, cfg.min_budget,
        rejected_norm = budget_to_normalized(rejected_budget, cfg.min_budget,

        pairs.append({
            "question": q,
            "preferred_budget": preferred_budget,
            "rejected_budget": rejected_budget,
            "preferred_norm": preferred_norm,

```

```

        "rejected_norm": rejected_norm,
    })

    return pairs

def train_dpo(train_data, test_data, featurizer, policy, llm, cfg,
    ↪oracle_records):
    """
    DPO training: Direct Preference Optimization.

    Loss:  $-\log(\text{sigmoid}(\beta * (\log_{\pi}(\text{preferred}) - \log_{\pi}(\text{rejected}))))$ 

    No value network. No rollouts. No advantage estimation. No NaN risk.
    Gradients come directly from preference pairs built from oracle data.
    """
    print("\nBuilding DPO preference pairs from oracle records...")
    pairs = build_dpo_pairs(oracle_records, cfg)
    print(f"    Total pairs: {len(pairs)}")
    if len(pairs) == 0:
        print("    ERROR: No valid pairs found. Check oracle records.")
        return

    # Show pair statistics
    pref_budgets = [p["preferred_budget"] for p in pairs]
    rej_budgets = [p["rejected_budget"] for p in pairs]
    print(f"    Preferred budgets: mean={sum(pref_budgets)/len(pref_budgets):.1f}, "
    ↪1f}, "
        f"min={min(pref_budgets)}, max={max(pref_budgets)}")
    print(f"    Rejected budgets: mean={sum(rej_budgets)/len(rej_budgets):.1f}, "
        f"min={min(rej_budgets)}, max={max(rej_budgets)}")

    # Pre-encode all question features
    print("    Pre-encoding question features...")
    all_questions = [p["question"] for p in pairs]
    all_feats = featurizer.encode_batch(all_questions)
    all_feats_tensor = torch.tensor(all_feats, dtype=torch.float32, device=cfg.
    ↪device)
    preferred_norms = torch.tensor(
        [p["preferred_norm"] for p in pairs], dtype=torch.float32, device=cfg.
    ↪device
    )
    rejected_norms = torch.tensor(
        [p["rejected_norm"] for p in pairs], dtype=torch.float32, device=cfg.
    ↪device
    )

```



```

optimizer = optim.Adam(policy.parameters(), lr=cfg.dpo_learning_rate)

best_acc = -1.0
best_reward = -1.0

print(f"\nStarting DPO training ({cfg.dpo_epochs} epochs, beta={cfg.
↪dpo_beta})...")
print(f"  {len(pairs)} preference pairs | batch_size={cfg.dpo_batch_size}")

for epoch in range(1, cfg.dpo_epochs + 1):
    policy.train()
    perm = torch.randperm(len(pairs))
    epoch_losses = []

    for start in range(0, len(pairs), cfg.dpo_batch_size):
        batch_idx = perm[start:start + cfg.dpo_batch_size]
        if len(batch_idx) < 2:
            continue

        batch_feats = all_feats_tensor[batch_idx]
        batch_pref = preferred_norms[batch_idx]
        batch_rej = rejected_norms[batch_idx]

        # Get policy log probs for preferred and rejected budgets
        mean, log_std = policy(batch_feats)
        std = torch.exp(log_std)
        dist = torch.distributions.Normal(mean.squeeze(-1), std.squeeze(-1))

        # Inverse sigmoid to get logit-space values for log_prob
        pref_raw = torch.logit(batch_pref.clamp(1e-6, 1 - 1e-6))
        rej_raw = torch.logit(batch_rej.clamp(1e-6, 1 - 1e-6))

        log_pi_preferred = dist.log_prob(pref_raw)
        log_pi_rejected = dist.log_prob(rej_raw)

        # DPO loss: push policy toward preferred, away from rejected
        # loss = -log(sigmoid(beta * (log_pi_pref - log_pi_rej)))
        logits = cfg.dpo_beta * (log_pi_preferred - log_pi_rejected)
        loss = -torch.nn.functional.logsigmoid(logits).mean()

        optimizer.zero_grad()
        loss.backward()
        torch.nn.utils.clip_grad_norm_(policy.parameters(), cfg.
↪grad_clip_norm)
        optimizer.step()

        epoch_losses.append(loss.item())

```

```

    avg_loss = sum(epoch_losses) / max(len(epoch_losses), 1)

    # Evaluate deterministic policy
    policy.eval()
    eval_result = evaluate_policy_deterministic(test_data, featurizer,
    ↪ policy, llm, cfg)
    acc = eval_result["accuracy"]
    avg_reward = eval_result["avg_reward"]
    avg_budget = eval_result["avg_budget"]
    std_budget = eval_result["std_budget"]

    print(f"\n{'='*80}")
    print(f"DPO Epoch {epoch}/{cfg.dpo_epochs}")
    print(f"{'='*80}")
    print(f"  Loss={avg_loss:.4f} | Acc={acc:.4f} | "
          f"AvgBudget={avg_budget:.1f} | StdBudget={std_budget:.1f}")

    if avg_reward > best_reward:
        best_reward = avg_reward
        torch.save(policy.state_dict(), cfg.best_dpo_path)
        print(f"  Saved best DPO model by reward = {best_reward:.4f}")

    if acc > best_acc:
        best_acc = acc
        torch.save(policy.state_dict(), cfg.best_dpo_acc_path)
        print(f"  Saved best DPO model by accuracy = {best_acc:.4f}")

    print("\nDPO training complete.")

def main():
    set_seed(CFG.seed)

    print("torch version:", torch.__version__)
    print("cuda available:", torch.cuda.is_available())
    if torch.cuda.is_available():
        print("gpu name:", torch.cuda.get_device_name(0))
        print("gpu memory:", torch.cuda.get_device_properties(0).total_memory //
    ↪ 1e9, "GB")

    if CFG.clear_cache_on_start:
        print(f"\nClearing cache: {CFG.cache_dir}")
        reset_cache_dir(CFG.cache_dir)
        for stale in ["oracle_labels_v11.json", "oracle_labels_v12.json",
                      "oracle_labels_v13.json", "oracle_labels_v14.json",
                      "oracle_labels_v15.json", "oracle_labels_v16.json",

```

```

        "oracle_labels_v17.json", CFG.oracle_cache_path]:
    if os.path.exists(stale):
        os.remove(stale)
        print(f"  Removed stale oracle cache: {stale}")
    else:
        ensure_dir(CFG.cache_dir)

    print(f"Using device: {CFG.device}")
    print(f"LLM batch size: {CFG.llm_batch_size}")

    # Load data
    print("\nLoading GSM8K...")
    ds = load_dataset("gsm8k", "main")
    train_data = list(ds["train"].select(range(min(CFG.train_subset,
    ↪len(ds["train"]))))))
    test_data = list(ds["test"].select(range(min(CFG.test_subset,
    ↪len(ds["test"]))))))
    print(f"Train: {len(train_data)} | Test: {len(test_data)}")

    # Load featurizer
    print("\nLoading featurizer...")
    featurizer = QuestionFeaturizer(CFG.embed_name, CFG.device)
    input_dim = featurizer.encode(train_data[0]["question"]).shape[0]
    print(f"Feature dim: {input_dim}")

    # Load policy only - DPO does not need a value network
    print("\nLoading policy network...")
    policy = ContinuousBudgetPolicy(input_dim=input_dim).to(CFG.device)
    with torch.no_grad():
        policy.mean_head.bias.fill_(0.85)  # init toward ~180 tokens

    # Load LLM
    print("\nLoading LLM...")
    llm = LLMReasoner(CFG.llm_name, CFG.device, batch_size=CFG.llm_batch_size)

    # Build oracle labels
    oracle_records = build_oracle_budget_labels(train_data, llm, CFG)

    # Warm-start policy on oracle budget labels
    warmstart_policy(oracle_records, test_data, featurizer, policy, llm, CFG)

    # Load best warm-start
    if os.path.exists(CFG.best_warmstart_acc_path):
        print("\nLoading best warm-start policy by accuracy...")
        policy.load_state_dict(torch.load(CFG.best_warmstart_acc_path,
    ↪map_location=CFG.device))
    elif os.path.exists(CFG.best_warmstart_path):

```

```

        print("\nLoading best warm-start policy by reward...")
        policy.load_state_dict(torch.load(CFG.best_warmstart_path,
↪map_location=CFG.device))

    print("\nPost warm-start evaluation:")
    warm_eval = evaluate_policy_deterministic(test_data, featurizer, policy,
↪llm, CFG)
    print(json.dumps({"warmstart_eval": warm_eval}, indent=2))

    # DPO Training
    train_dpo(
        train_data=train_data,
        test_data=test_data,
        featurizer=featurizer,
        policy=policy,
        llm=llm,
        cfg=CFG,
        oracle_records=oracle_records,
    )

    # Load best DPO model
    print("\nLoading best DPO model for final evaluation...")
    if os.path.exists(CFG.best_dpo_acc_path):
        policy.load_state_dict(torch.load(CFG.best_dpo_acc_path,
↪map_location=CFG.device))
    elif os.path.exists(CFG.best_dpo_path):
        policy.load_state_dict(torch.load(CFG.best_dpo_path, map_location=CFG.
↪device))

    # Final evaluation
    print("\nFinal deterministic policy evaluation:")
    final_eval = evaluate_policy_deterministic(test_data, featurizer, policy,
↪llm, CFG)
    print(json.dumps({"final_deterministic_eval": final_eval}, indent=2))

    # Save final models
    torch.save(policy.state_dict(), CFG.final_policy_path)
    print(f"Saved: {CFG.final_policy_path}")

    # Final results table - run fixed baselines at end for comparison
    print("\n" + "="*72)
    print("FINAL RESULTS COMPARISON")
    print("="*72)
    print(f"{'Method':<40} {'Acc':>6} {'AvgTokens':>9} {'AvgReward':>9}")
    print("-"*72)
    for b in CFG.baseline_budgets:
        res = evaluate_fixed_budget(test_data, llm, CFG, b)

```

```

        print(f" Fixed budget {b:<4}                                "
              f"{res['accuracy']:.4f} {res['avg_thinking_tokens']:>9.0f} "
              f"{res['avg_reward']:>9.4f}")
    print(f" Policy (DPO deterministic)                                "
          f"{final_eval['accuracy']:.4f} {final_eval['avg_thinking_tokens']:>9.
↪2f}  "
          f"{final_eval['avg_reward']:>9.4f}")
    print("="*72)
    print("\nAI Disclosure: Written with Claude's assistance")

main()

```

/opt/conda/lib/python3.13/site-packages/tqdm/auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html

```
from .autonotebook import tqdm as notebook_tqdm
```

torch version: 2.11.0+cu128

cuda available: True

gpu name: NVIDIA H200 NVL

gpu memory: 150.0 GB

Clearing cache: ./cache_dpo_v17

Removed stale oracle cache: oracle_labels_v14.json

Removed stale oracle cache: oracle_labels_v15.json

Removed stale oracle cache: oracle_labels_v16.json

Removed stale oracle cache: oracle_labels_v17.json

Using device: cuda

LLM batch size: 8

Loading GSM8K...

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

Train: 300 | Test: 100

Loading featurizer...

Loading weights: 100%| | 103/103 [00:00<00:00, 10329.81it/s]

BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2

Key	Status		
-----+-----+-----			
embeddings.position_ids	UNEXPECTED		

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.

Feature dim: 446

Loading policy network...

Loading LLM...

Loading weights: 100%| | 339/339 [00:28<00:00, 11.76it/s]

Building oracle labels with BATCHED GPU inference...

Trying budget=128 for all 300 questions...

Budget 128 done in 222.1s | Correct: 198/300 (66.0%)

Trying budget=144 for all 300 questions...

Budget 144 done in 123.3s | Correct: 213/300 (71.0%)

Trying budget=160 for all 300 questions...

Budget 160 done in 119.2s | Correct: 221/300 (73.7%)

Trying budget=176 for all 300 questions...

Budget 176 done in 138.8s | Correct: 237/300 (79.0%)

Trying budget=192 for all 300 questions...

Budget 192 done in 141.7s | Correct: 247/300 (82.3%)

Trying budget=210 for all 300 questions...

Budget 210 done in 157.9s | Correct: 256/300 (85.3%)

Trying budget=220 for all 300 questions...

Budget 220 done in 153.0s | Correct: 262/300 (87.3%)

Oracle stats: mean=140.1, std=23.8, solvable=0.943

budget=128: 215 questions (71.7%)

budget=144: 31 questions (10.3%)

budget=160: 12 questions (4.0%)

budget=176: 16 questions (5.3%)

budget=192: 13 questions (4.3%)

budget=210: 4 questions (1.3%)

budget=220: 9 questions (3.0%)

Saved oracle labels to oracle_labels_v17.json

Starting supervised warm-start...

[Warmstart 1/10] Step 20/300 | Loss=0.1763 | PredAvg=139.4 | TargetAvg=138.4 | MAE=37.70

[Warmstart 1/10] Step 40/300 | Loss=0.1083 | PredAvg=136.8 | TargetAvg=136.0 | MAE=28.00

[Warmstart 1/10] Step 60/300 | Loss=0.0834 | PredAvg=135.0 | TargetAvg=134.1 | MAE=25.27

[Warmstart 1/10] Step 80/300 | Loss=0.1011 | PredAvg=132.9 | TargetAvg=136.6 | MAE=27.19

[Warmstart 1/10] Step 100/300 | Loss=0.1136 | PredAvg=135.5 | TargetAvg=137.6 | MAE=28.34

[Warmstart 1/10] Step 120/300 | Loss=0.1116 | PredAvg=135.8 | TargetAvg=137.5 | MAE=27.83

[Warmstart 1/10] Step 140/300 | Loss=0.1560 | PredAvg=135.1 | TargetAvg=138.9 |

MAE=29.33
 [Warmstart 1/10] Step 160/300 | Loss=0.1630 | PredAvg=133.5 | TargetAvg=139.3 |
 MAE=29.56
 [Warmstart 1/10] Step 180/300 | Loss=0.1585 | PredAvg=135.3 | TargetAvg=139.4 |
 MAE=30.05
 [Warmstart 1/10] Step 200/300 | Loss=0.1488 | PredAvg=135.3 | TargetAvg=139.3 |
 MAE=29.14
 [Warmstart 1/10] Step 220/300 | Loss=0.1681 | PredAvg=135.8 | TargetAvg=140.3 |
 MAE=30.14
 [Warmstart 1/10] Step 240/300 | Loss=0.1628 | PredAvg=136.0 | TargetAvg=140.4 |
 MAE=30.14
 [Warmstart 1/10] Step 260/300 | Loss=0.1604 | PredAvg=136.2 | TargetAvg=140.5 |
 MAE=30.13
 [Warmstart 1/10] Step 280/300 | Loss=0.1643 | PredAvg=135.4 | TargetAvg=140.5 |
 MAE=29.92
 [Warmstart 1/10] Step 300/300 | Loss=0.1604 | PredAvg=135.2 | TargetAvg=140.1 |
 MAE=29.32
 Warmstart epoch 1 done | Loss=0.1604
 Det eval complete | Acc=0.6000 | AvgBudget=134.2 | StdBudget=17.4 |
 Range=[100, 171]
 Saved best warmstart by reward = 0.6829
 Saved best warmstart by accuracy = 0.6000
 [Warmstart 2/10] Step 20/300 | Loss=0.2370 | PredAvg=140.2 | TargetAvg=142.2 |
 MAE=31.45
 [Warmstart 2/10] Step 40/300 | Loss=0.2098 | PredAvg=137.4 | TargetAvg=143.9 |
 MAE=30.07
 [Warmstart 2/10] Step 60/300 | Loss=0.1615 | PredAvg=136.9 | TargetAvg=138.9 |
 MAE=29.18
 [Warmstart 2/10] Step 80/300 | Loss=0.1306 | PredAvg=134.2 | TargetAvg=137.2 |
 MAE=26.51
 [Warmstart 2/10] Step 100/300 | Loss=0.1257 | PredAvg=134.1 | TargetAvg=137.2 |
 MAE=25.37
 [Warmstart 2/10] Step 120/300 | Loss=0.1261 | PredAvg=134.7 | TargetAvg=138.1 |
 MAE=24.97
 [Warmstart 2/10] Step 140/300 | Loss=0.1135 | PredAvg=135.1 | TargetAvg=137.2 |
 MAE=24.23
 [Warmstart 2/10] Step 160/300 | Loss=0.1132 | PredAvg=133.9 | TargetAvg=136.9 |
 MAE=23.91
 [Warmstart 2/10] Step 180/300 | Loss=0.1110 | PredAvg=134.9 | TargetAvg=137.5 |
 MAE=24.08
 [Warmstart 2/10] Step 200/300 | Loss=0.1263 | PredAvg=135.6 | TargetAvg=139.1 |
 MAE=25.09
 [Warmstart 2/10] Step 220/300 | Loss=0.1234 | PredAvg=135.1 | TargetAvg=138.9 |
 MAE=24.81
 [Warmstart 2/10] Step 240/300 | Loss=0.1347 | PredAvg=135.3 | TargetAvg=140.2 |
 MAE=25.10
 [Warmstart 2/10] Step 260/300 | Loss=0.1329 | PredAvg=136.9 | TargetAvg=140.6 |
 MAE=25.35

[Warmstart 2/10] Step 280/300 | Loss=0.1340 | PredAvg=136.2 | TargetAvg=140.6 |
 MAE=25.33
 [Warmstart 2/10] Step 300/300 | Loss=0.1275 | PredAvg=136.0 | TargetAvg=140.1 |
 MAE=24.52
 Warmstart epoch 2 done | Loss=0.1275
 Det eval complete | Acc=0.6500 | AvgBudget=134.6 | StdBudget=15.8 |
 Range=[102, 167]
 Saved best warmstart by reward = 0.7452
 Saved best warmstart by accuracy = 0.6500
 [Warmstart 3/10] Step 20/300 | Loss=0.0823 | PredAvg=140.8 | TargetAvg=139.3 |
 MAE=20.10
 [Warmstart 3/10] Step 40/300 | Loss=0.0794 | PredAvg=135.2 | TargetAvg=138.4 |
 MAE=19.88
 [Warmstart 3/10] Step 60/300 | Loss=0.1036 | PredAvg=134.8 | TargetAvg=138.9 |
 MAE=20.60
 [Warmstart 3/10] Step 80/300 | Loss=0.1508 | PredAvg=133.9 | TargetAvg=141.2 |
 MAE=24.20
 [Warmstart 3/10] Step 100/300 | Loss=0.1365 | PredAvg=135.7 | TargetAvg=140.6 |
 MAE=23.71
 [Warmstart 3/10] Step 120/300 | Loss=0.1236 | PredAvg=134.8 | TargetAvg=140.1 |
 MAE=23.17
 [Warmstart 3/10] Step 140/300 | Loss=0.1176 | PredAvg=135.6 | TargetAvg=139.5 |
 MAE=22.40
 [Warmstart 3/10] Step 160/300 | Loss=0.1085 | PredAvg=134.8 | TargetAvg=138.3 |
 MAE=21.69
 [Warmstart 3/10] Step 180/300 | Loss=0.1127 | PredAvg=134.4 | TargetAvg=138.9 |
 MAE=21.93
 [Warmstart 3/10] Step 200/300 | Loss=0.1150 | PredAvg=135.8 | TargetAvg=139.6 |
 MAE=22.41
 [Warmstart 3/10] Step 220/300 | Loss=0.1163 | PredAvg=135.1 | TargetAvg=139.4 |
 MAE=22.08
 [Warmstart 3/10] Step 240/300 | Loss=0.1218 | PredAvg=134.5 | TargetAvg=139.2 |
 MAE=21.67
 [Warmstart 3/10] Step 260/300 | Loss=0.1178 | PredAvg=134.9 | TargetAvg=140.0 |
 MAE=21.59
 [Warmstart 3/10] Step 280/300 | Loss=0.1173 | PredAvg=136.6 | TargetAvg=140.0 |
 MAE=22.39
 [Warmstart 3/10] Step 300/300 | Loss=0.1193 | PredAvg=135.7 | TargetAvg=140.1 |
 MAE=22.38
 Warmstart epoch 3 done | Loss=0.1193
 Det eval complete | Acc=0.6500 | AvgBudget=142.9 | StdBudget=11.5 |
 Range=[113, 169]
 [Warmstart 4/10] Step 20/300 | Loss=0.1649 | PredAvg=134.4 | TargetAvg=139.8 |
 MAE=25.55
 [Warmstart 4/10] Step 40/300 | Loss=0.1088 | PredAvg=132.6 | TargetAvg=138.3 |
 MAE=21.02
 [Warmstart 4/10] Step 60/300 | Loss=0.1140 | PredAvg=135.8 | TargetAvg=140.7 |
 MAE=22.88

[Warmstart 4/10] Step 80/300 | Loss=0.1275 | PredAvg=135.1 | TargetAvg=143.1 | MAE=23.40
 [Warmstart 4/10] Step 100/300 | Loss=0.1289 | PredAvg=134.9 | TargetAvg=143.1 | MAE=22.50
 [Warmstart 4/10] Step 120/300 | Loss=0.1216 | PredAvg=134.1 | TargetAvg=141.9 | MAE=21.02
 [Warmstart 4/10] Step 140/300 | Loss=0.1051 | PredAvg=133.9 | TargetAvg=140.4 | MAE=19.29
 [Warmstart 4/10] Step 160/300 | Loss=0.1177 | PredAvg=133.9 | TargetAvg=140.8 | MAE=19.56
 [Warmstart 4/10] Step 180/300 | Loss=0.1140 | PredAvg=133.1 | TargetAvg=140.6 | MAE=19.31
 [Warmstart 4/10] Step 200/300 | Loss=0.1111 | PredAvg=134.8 | TargetAvg=140.6 | MAE=19.89
 [Warmstart 4/10] Step 220/300 | Loss=0.1193 | PredAvg=134.2 | TargetAvg=140.6 | MAE=20.00
 [Warmstart 4/10] Step 240/300 | Loss=0.1189 | PredAvg=133.6 | TargetAvg=140.2 | MAE=19.52
 [Warmstart 4/10] Step 260/300 | Loss=0.1223 | PredAvg=133.6 | TargetAvg=140.8 | MAE=19.98
 [Warmstart 4/10] Step 280/300 | Loss=0.1140 | PredAvg=133.2 | TargetAvg=140.0 | MAE=19.09
 [Warmstart 4/10] Step 300/300 | Loss=0.1185 | PredAvg=133.2 | TargetAvg=140.1 | MAE=19.19
 Warmstart epoch 4 done | Loss=0.1185
 Det eval complete | Acc=0.5900 | AvgBudget=130.3 | StdBudget=7.9 | Range=[112, 154]
 [Warmstart 5/10] Step 20/300 | Loss=0.0042 | PredAvg=126.9 | TargetAvg=128.8 | MAE=5.90
 [Warmstart 5/10] Step 40/300 | Loss=0.0482 | PredAvg=130.7 | TargetAvg=132.8 | MAE=11.60
 [Warmstart 5/10] Step 60/300 | Loss=0.0783 | PredAvg=132.3 | TargetAvg=136.0 | MAE=14.92
 [Warmstart 5/10] Step 80/300 | Loss=0.1034 | PredAvg=131.5 | TargetAvg=137.7 | MAE=16.06
 [Warmstart 5/10] Step 100/300 | Loss=0.0920 | PredAvg=132.2 | TargetAvg=137.7 | MAE=16.42
 [Warmstart 5/10] Step 120/300 | Loss=0.0925 | PredAvg=133.7 | TargetAvg=138.8 | MAE=16.93
 [Warmstart 5/10] Step 140/300 | Loss=0.0983 | PredAvg=134.0 | TargetAvg=139.3 | MAE=17.56
 [Warmstart 5/10] Step 160/300 | Loss=0.1071 | PredAvg=134.0 | TargetAvg=140.7 | MAE=18.19
 [Warmstart 5/10] Step 180/300 | Loss=0.0988 | PredAvg=134.0 | TargetAvg=140.0 | MAE=17.64
 [Warmstart 5/10] Step 200/300 | Loss=0.1081 | PredAvg=134.6 | TargetAvg=141.1 | MAE=19.07
 [Warmstart 5/10] Step 220/300 | Loss=0.1050 | PredAvg=135.2 | TargetAvg=141.0 |

MAE=18.77
 [Warmstart 5/10] Step 240/300 | Loss=0.1075 | PredAvg=134.7 | TargetAvg=140.8 |
 MAE=18.73
 [Warmstart 5/10] Step 260/300 | Loss=0.1036 | PredAvg=134.2 | TargetAvg=140.5 |
 MAE=18.28
 [Warmstart 5/10] Step 280/300 | Loss=0.1036 | PredAvg=133.8 | TargetAvg=140.4 |
 MAE=18.10
 [Warmstart 5/10] Step 300/300 | Loss=0.1022 | PredAvg=134.1 | TargetAvg=140.1 |
 MAE=18.15
 Warmstart epoch 5 done | Loss=0.1022
 Det eval complete | Acc=0.5600 | AvgBudget=119.9 | StdBudget=6.4 | Range=[107,
 142]
 [Warmstart 6/10] Step 20/300 | Loss=0.1721 | PredAvg=128.8 | TargetAvg=140.7 |
 MAE=16.85
 [Warmstart 6/10] Step 40/300 | Loss=0.1060 | PredAvg=129.5 | TargetAvg=137.6 |
 MAE=13.72
 [Warmstart 6/10] Step 60/300 | Loss=0.1136 | PredAvg=129.7 | TargetAvg=138.9 |
 MAE=14.87
 [Warmstart 6/10] Step 80/300 | Loss=0.0875 | PredAvg=130.9 | TargetAvg=137.6 |
 MAE=12.81
 [Warmstart 6/10] Step 100/300 | Loss=0.1044 | PredAvg=131.9 | TargetAvg=139.8 |
 MAE=14.27
 [Warmstart 6/10] Step 120/300 | Loss=0.1060 | PredAvg=133.4 | TargetAvg=139.9 |
 MAE=15.25
 [Warmstart 6/10] Step 140/300 | Loss=0.1001 | PredAvg=133.2 | TargetAvg=139.9 |
 MAE=15.23
 [Warmstart 6/10] Step 160/300 | Loss=0.0923 | PredAvg=132.9 | TargetAvg=139.6 |
 MAE=14.91
 [Warmstart 6/10] Step 180/300 | Loss=0.1045 | PredAvg=132.5 | TargetAvg=140.2 |
 MAE=15.68
 [Warmstart 6/10] Step 200/300 | Loss=0.0960 | PredAvg=132.3 | TargetAvg=139.3 |
 MAE=14.95
 [Warmstart 6/10] Step 220/300 | Loss=0.0948 | PredAvg=132.2 | TargetAvg=139.5 |
 MAE=15.18
 [Warmstart 6/10] Step 240/300 | Loss=0.0972 | PredAvg=132.1 | TargetAvg=139.6 |
 MAE=15.44
 [Warmstart 6/10] Step 260/300 | Loss=0.0989 | PredAvg=133.7 | TargetAvg=139.9 |
 MAE=16.28
 [Warmstart 6/10] Step 280/300 | Loss=0.1061 | PredAvg=133.2 | TargetAvg=140.3 |
 MAE=16.52
 [Warmstart 6/10] Step 300/300 | Loss=0.1043 | PredAvg=133.6 | TargetAvg=140.1 |
 MAE=16.43
 Warmstart epoch 6 done | Loss=0.1043
 Det eval complete | Acc=0.6300 | AvgBudget=128.2 | StdBudget=6.8 | Range=[112,
 149]
 [Warmstart 7/10] Step 20/300 | Loss=0.0148 | PredAvg=129.8 | TargetAvg=132.0 |
 MAE=7.55
 [Warmstart 7/10] Step 40/300 | Loss=0.0254 | PredAvg=130.5 | TargetAvg=134.8 |

MAE=10.35
 [Warmstart 7/10] Step 60/300 | Loss=0.0507 | PredAvg=130.3 | TargetAvg=135.7 |
 MAE=11.80
 [Warmstart 7/10] Step 80/300 | Loss=0.0489 | PredAvg=133.1 | TargetAvg=135.9 |
 MAE=12.55
 [Warmstart 7/10] Step 100/300 | Loss=0.0763 | PredAvg=132.9 | TargetAvg=138.3 |
 MAE=14.97
 [Warmstart 7/10] Step 120/300 | Loss=0.0822 | PredAvg=132.3 | TargetAvg=138.4 |
 MAE=15.07
 [Warmstart 7/10] Step 140/300 | Loss=0.0888 | PredAvg=132.1 | TargetAvg=138.5 |
 MAE=15.27
 [Warmstart 7/10] Step 160/300 | Loss=0.0830 | PredAvg=131.7 | TargetAvg=138.1 |
 MAE=14.64
 [Warmstart 7/10] Step 180/300 | Loss=0.0963 | PredAvg=132.8 | TargetAvg=140.0 |
 MAE=15.99
 [Warmstart 7/10] Step 200/300 | Loss=0.0882 | PredAvg=132.8 | TargetAvg=139.3 |
 MAE=15.24
 [Warmstart 7/10] Step 220/300 | Loss=0.0918 | PredAvg=133.0 | TargetAvg=139.9 |
 MAE=15.53
 [Warmstart 7/10] Step 240/300 | Loss=0.0970 | PredAvg=133.0 | TargetAvg=140.4 |
 MAE=16.07
 [Warmstart 7/10] Step 260/300 | Loss=0.0958 | PredAvg=133.2 | TargetAvg=140.2 |
 MAE=15.85
 [Warmstart 7/10] Step 280/300 | Loss=0.0972 | PredAvg=133.4 | TargetAvg=140.3 |
 MAE=15.91
 [Warmstart 7/10] Step 300/300 | Loss=0.0965 | PredAvg=133.0 | TargetAvg=140.1 |
 MAE=15.82
 Warmstart epoch 7 done | Loss=0.0965
 Det eval complete | Acc=0.6800 | AvgBudget=144.6 | StdBudget=7.7 | Range=[131, 165]
 Saved best warmstart by reward = 0.7777
 Saved best warmstart by accuracy = 0.6800
 [Warmstart 8/10] Step 20/300 | Loss=0.0699 | PredAvg=152.8 | TargetAvg=147.8 |
 MAE=21.75
 [Warmstart 8/10] Step 40/300 | Loss=0.0569 | PredAvg=139.9 | TargetAvg=141.5 |
 MAE=16.60
 [Warmstart 8/10] Step 60/300 | Loss=0.0498 | PredAvg=137.2 | TargetAvg=138.9 |
 MAE=14.58
 [Warmstart 8/10] Step 80/300 | Loss=0.0449 | PredAvg=136.1 | TargetAvg=138.2 |
 MAE=13.79
 [Warmstart 8/10] Step 100/300 | Loss=0.0463 | PredAvg=135.4 | TargetAvg=138.4 |
 MAE=14.32
 [Warmstart 8/10] Step 120/300 | Loss=0.0835 | PredAvg=135.0 | TargetAvg=140.5 |
 MAE=15.98
 [Warmstart 8/10] Step 140/300 | Loss=0.0766 | PredAvg=134.6 | TargetAvg=140.0 |
 MAE=15.50
 [Warmstart 8/10] Step 160/300 | Loss=0.0788 | PredAvg=134.3 | TargetAvg=139.8 |
 MAE=15.26

[Warmstart 8/10] Step 180/300 | Loss=0.0778 | PredAvg=133.8 | TargetAvg=139.5 |
 MAE=14.66
 [Warmstart 8/10] Step 200/300 | Loss=0.0739 | PredAvg=133.8 | TargetAvg=138.9 |
 MAE=14.40
 [Warmstart 8/10] Step 220/300 | Loss=0.0776 | PredAvg=133.5 | TargetAvg=139.2 |
 MAE=14.67
 [Warmstart 8/10] Step 240/300 | Loss=0.0807 | PredAvg=133.6 | TargetAvg=139.3 |
 MAE=14.61
 [Warmstart 8/10] Step 260/300 | Loss=0.0820 | PredAvg=133.6 | TargetAvg=139.6 |
 MAE=14.77
 [Warmstart 8/10] Step 280/300 | Loss=0.0970 | PredAvg=133.6 | TargetAvg=140.2 |
 MAE=15.41
 [Warmstart 8/10] Step 300/300 | Loss=0.0966 | PredAvg=133.6 | TargetAvg=140.1 |
 MAE=15.31
 Warmstart epoch 8 done | Loss=0.0966
 Det eval complete | Acc=0.6300 | AvgBudget=126.5 | StdBudget=6.1 | Range=[114,
 145]
 [Warmstart 9/10] Step 20/300 | Loss=0.0897 | PredAvg=127.8 | TargetAvg=140.0 |
 MAE=14.35
 [Warmstart 9/10] Step 40/300 | Loss=0.0662 | PredAvg=129.5 | TargetAvg=138.4 |
 MAE=12.78
 [Warmstart 9/10] Step 60/300 | Loss=0.1292 | PredAvg=131.4 | TargetAvg=143.4 |
 MAE=16.58
 [Warmstart 9/10] Step 80/300 | Loss=0.1190 | PredAvg=133.6 | TargetAvg=144.1 |
 MAE=17.02
 [Warmstart 9/10] Step 100/300 | Loss=0.1019 | PredAvg=133.8 | TargetAvg=142.6 |
 MAE=16.36
 [Warmstart 9/10] Step 120/300 | Loss=0.1013 | PredAvg=133.6 | TargetAvg=141.9 |
 MAE=16.40
 [Warmstart 9/10] Step 140/300 | Loss=0.1064 | PredAvg=133.8 | TargetAvg=142.8 |
 MAE=17.16
 [Warmstart 9/10] Step 160/300 | Loss=0.1088 | PredAvg=133.7 | TargetAvg=142.9 |
 MAE=17.09
 [Warmstart 9/10] Step 180/300 | Loss=0.1078 | PredAvg=134.2 | TargetAvg=142.7 |
 MAE=16.93
 [Warmstart 9/10] Step 200/300 | Loss=0.1124 | PredAvg=133.9 | TargetAvg=142.6 |
 MAE=16.86
 [Warmstart 9/10] Step 220/300 | Loss=0.1060 | PredAvg=133.6 | TargetAvg=141.9 |
 MAE=16.35
 [Warmstart 9/10] Step 240/300 | Loss=0.1045 | PredAvg=133.4 | TargetAvg=141.4 |
 MAE=15.87
 [Warmstart 9/10] Step 260/300 | Loss=0.0988 | PredAvg=132.9 | TargetAvg=140.7 |
 MAE=15.22
 [Warmstart 9/10] Step 280/300 | Loss=0.0934 | PredAvg=133.1 | TargetAvg=140.5 |
 MAE=14.98
 [Warmstart 9/10] Step 300/300 | Loss=0.0934 | PredAvg=133.0 | TargetAvg=140.1 |
 MAE=14.73
 Warmstart epoch 9 done | Loss=0.0934

```

    Det eval complete | Acc=0.6200 | AvgBudget=125.7 | StdBudget=6.0 | Range=[114,
144]
[Warmstart 10/10] Step 20/300 | Loss=0.0062 | PredAvg=130.8 | TargetAvg=129.6 |
MAE=7.50
[Warmstart 10/10] Step 40/300 | Loss=0.0478 | PredAvg=130.2 | TargetAvg=133.1 |
MAE=9.38
[Warmstart 10/10] Step 60/300 | Loss=0.0704 | PredAvg=134.0 | TargetAvg=137.7 |
MAE=13.28
[Warmstart 10/10] Step 80/300 | Loss=0.0802 | PredAvg=132.7 | TargetAvg=137.7 |
MAE=13.55
[Warmstart 10/10] Step 100/300 | Loss=0.0751 | PredAvg=132.5 | TargetAvg=137.8 |
MAE=13.11
[Warmstart 10/10] Step 120/300 | Loss=0.0805 | PredAvg=134.8 | TargetAvg=139.2 |
MAE=14.78
[Warmstart 10/10] Step 140/300 | Loss=0.0783 | PredAvg=134.3 | TargetAvg=138.8 |
MAE=14.38
[Warmstart 10/10] Step 160/300 | Loss=0.0711 | PredAvg=133.7 | TargetAvg=138.1 |
MAE=13.74
[Warmstart 10/10] Step 180/300 | Loss=0.0729 | PredAvg=134.1 | TargetAvg=138.2 |
MAE=13.96
[Warmstart 10/10] Step 200/300 | Loss=0.0720 | PredAvg=134.1 | TargetAvg=138.7 |
MAE=14.13
[Warmstart 10/10] Step 220/300 | Loss=0.0676 | PredAvg=134.1 | TargetAvg=138.2 |
MAE=13.81
[Warmstart 10/10] Step 240/300 | Loss=0.0812 | PredAvg=134.2 | TargetAvg=139.3 |
MAE=14.65
[Warmstart 10/10] Step 260/300 | Loss=0.0860 | PredAvg=134.6 | TargetAvg=140.1 |
MAE=15.36
[Warmstart 10/10] Step 280/300 | Loss=0.0886 | PredAvg=134.7 | TargetAvg=140.3 |
MAE=15.64
[Warmstart 10/10] Step 300/300 | Loss=0.0878 | PredAvg=134.6 | TargetAvg=140.1 |
MAE=15.57
Warmstart epoch 10 done | Loss=0.0878
    Det eval complete | Acc=0.6400 | AvgBudget=128.8 | StdBudget=6.3 | Range=[112,
148]

```

Loading best warm-start policy by accuracy...

Post warm-start evaluation:

```

    Det eval complete | Acc=0.6800 | AvgBudget=144.6 | StdBudget=7.7 | Range=[131,
165]
{
  "warmstart_eval": {
    "accuracy": 0.68,
    "avg_thinking_tokens": 144.64,
    "avg_reward": 0.7776800000000001,
    "avg_budget": 144.64,
    "min_budget": 131.0,

```

```

    "max_budget": 165.0,
    "std_budget": 7.73759652605381
  }
}

```

Building DPO preference pairs from oracle records...

```

Total pairs: 74
Preferred budgets: mean=139.1, min=128, max=210
Rejected budgets: mean=188.5, min=144, max=220
Pre-encoding question features...

```

Starting DPO training (20 epochs, beta=0.1)...

```

74 preference pairs | batch_size=32
Det eval complete | Acc=0.6700 | AvgBudget=141.9 | StdBudget=7.5 | Range=[129,
162]

```

```

=====
DPO Epoch 1/20
=====

```

```

Loss=0.6765 | Acc=0.6700 | AvgBudget=141.9 | StdBudget=7.5
Saved best DPO model by reward = 0.7666
Saved best DPO model by accuracy = 0.6700
Det eval complete | Acc=0.6700 | AvgBudget=139.2 | StdBudget=7.2 | Range=[127,
160]

```

```

=====
DPO Epoch 2/20
=====

```

```

Loss=0.6758 | Acc=0.6700 | AvgBudget=139.2 | StdBudget=7.2
Saved best DPO model by reward = 0.7679
Det eval complete | Acc=0.6700 | AvgBudget=136.6 | StdBudget=6.9 | Range=[125,
157]

```

```

=====
DPO Epoch 3/20
=====

```

```

Loss=0.6754 | Acc=0.6700 | AvgBudget=136.6 | StdBudget=6.9
Saved best DPO model by reward = 0.7692
Det eval complete | Acc=0.6600 | AvgBudget=133.9 | StdBudget=6.6 | Range=[122,
153]

```

```

=====
DPO Epoch 4/20
=====

```

```

Loss=0.6724 | Acc=0.6600 | AvgBudget=133.9 | StdBudget=6.6
Det eval complete | Acc=0.6700 | AvgBudget=131.4 | StdBudget=6.4 | Range=[119,
150]

```

=====

DP0 Epoch 5/20

=====

Loss=0.6695 | Acc=0.6700 | AvgBudget=131.4 | StdBudget=6.4
Saved best DP0 model by reward = 0.7718
Det eval complete | Acc=0.6600 | AvgBudget=128.9 | StdBudget=6.2 | Range=[116,
147]

=====

DP0 Epoch 6/20

=====

Loss=0.6634 | Acc=0.6600 | AvgBudget=128.9 | StdBudget=6.2
Det eval complete | Acc=0.5800 | AvgBudget=126.5 | StdBudget=6.1 | Range=[113,
144]

=====

DP0 Epoch 7/20

=====

Loss=0.6657 | Acc=0.5800 | AvgBudget=126.5 | StdBudget=6.1
Det eval complete | Acc=0.6300 | AvgBudget=124.2 | StdBudget=6.0 | Range=[111,
142]

=====

DP0 Epoch 8/20

=====

Loss=0.6571 | Acc=0.6300 | AvgBudget=124.2 | StdBudget=6.0
Det eval complete | Acc=0.5700 | AvgBudget=122.1 | StdBudget=5.9 | Range=[109,
140]

=====

DP0 Epoch 9/20

=====

Loss=0.6591 | Acc=0.5700 | AvgBudget=122.1 | StdBudget=5.9
Det eval complete | Acc=0.5700 | AvgBudget=120.1 | StdBudget=5.9 | Range=[107,
138]

=====

DP0 Epoch 10/20

=====

Loss=0.6559 | Acc=0.5700 | AvgBudget=120.1 | StdBudget=5.9
Det eval complete | Acc=0.5800 | AvgBudget=118.2 | StdBudget=5.9 | Range=[106,
137]

=====

DP0 Epoch 11/20

=====

Loss=0.6501 | Acc=0.5800 | AvgBudget=118.2 | StdBudget=5.9
Det eval complete | Acc=0.5900 | AvgBudget=116.3 | StdBudget=5.9 | Range=[104,

135]

=====
DP0 Epoch 12/20
=====

Loss=0.6491 | Acc=0.5900 | AvgBudget=116.3 | StdBudget=5.9
Det eval complete | Acc=0.6200 | AvgBudget=114.6 | StdBudget=5.9 | Range=[103,
134]

=====
DP0 Epoch 13/20
=====

Loss=0.6441 | Acc=0.6200 | AvgBudget=114.6 | StdBudget=5.9
Det eval complete | Acc=0.5900 | AvgBudget=113.0 | StdBudget=5.8 | Range=[101,
132]

=====
DP0 Epoch 14/20
=====

Loss=0.6459 | Acc=0.5900 | AvgBudget=113.0 | StdBudget=5.8
Det eval complete | Acc=0.5900 | AvgBudget=111.5 | StdBudget=5.8 | Range=[100,
131]

=====
DP0 Epoch 15/20
=====

Loss=0.6419 | Acc=0.5900 | AvgBudget=111.5 | StdBudget=5.8
Det eval complete | Acc=0.6400 | AvgBudget=110.2 | StdBudget=5.7 | Range=[100,
129]

=====
DP0 Epoch 16/20
=====

Loss=0.6415 | Acc=0.6400 | AvgBudget=110.2 | StdBudget=5.7
Det eval complete | Acc=0.6500 | AvgBudget=109.0 | StdBudget=5.6 | Range=[99,
128]

=====
DP0 Epoch 17/20
=====

Loss=0.6401 | Acc=0.6500 | AvgBudget=109.0 | StdBudget=5.6
Det eval complete | Acc=0.6000 | AvgBudget=107.8 | StdBudget=5.4 | Range=[98,
127]

=====
DP0 Epoch 18/20
=====

Loss=0.6306 | Acc=0.6000 | AvgBudget=107.8 | StdBudget=5.4

Det eval complete | Acc=0.5900 | AvgBudget=106.7 | StdBudget=5.3 | Range=[98, 125]

=====

DPO Epoch 19/20

=====

Loss=0.6277 | Acc=0.5900 | AvgBudget=106.7 | StdBudget=5.3
Det eval complete | Acc=0.5800 | AvgBudget=105.7 | StdBudget=5.2 | Range=[97, 124]

=====

DPO Epoch 20/20

=====

Loss=0.6268 | Acc=0.5800 | AvgBudget=105.7 | StdBudget=5.2

DPO training complete.

Loading best DPO model for final evaluation...

Final deterministic policy evaluation:

Det eval complete | Acc=0.6700 | AvgBudget=141.9 | StdBudget=7.5 | Range=[129, 162]

```
{
  "final_deterministic_eval": {
    "accuracy": 0.67,
    "avg_thinking_tokens": 141.87,
    "avg_reward": 0.766565,
    "avg_budget": 141.87,
    "min_budget": 129.0,
    "max_budget": 162.0,
    "std_budget": 7.498873248695433
  }
}
```

Saved: final_policy_v17.pt

=====

FINAL RESULTS COMPARISON

=====

Method	Acc	AvgTokens	AvgReward
--------	-----	-----------	-----------

Evaluating fixed budget = 96...

Fixed 96 40/100 Acc=0.4750 AvgReward=0.5457			
Fixed 96 80/100 Acc=0.4625 AvgReward=0.5301			
Fixed 96 100/100 Acc=0.4700 AvgReward=0.5395			
Fixed budget 96	0.4700	96	0.5395

Evaluating fixed budget = 128...

Fixed 128 40/100 Acc=0.5500 AvgReward=0.6235			
Fixed 128 80/100 Acc=0.6250 AvgReward=0.7172			
Fixed 128 100/100 Acc=0.6300 AvgReward=0.7235			
Fixed budget 128	0.6300	128	0.7235

Evaluating fixed budget = 160...

Fixed 160 40/100 Acc=0.6750 AvgReward=0.7637			
Fixed 160 80/100 Acc=0.7250 AvgReward=0.8262			
Fixed 160 100/100 Acc=0.7500 AvgReward=0.8575			
Fixed budget 160	0.7500	160	0.8575

Evaluating fixed budget = 192...

Fixed 192 40/100 Acc=0.6500 AvgReward=0.7165			
Fixed 192 80/100 Acc=0.7250 AvgReward=0.8102			
Fixed 192 100/100 Acc=0.7500 AvgReward=0.8415			
Fixed budget 192	0.7500	192	0.8415

Evaluating fixed budget = 224...

Fixed 224 40/100 Acc=0.7500 AvgReward=0.8255			
Fixed 224 80/100 Acc=0.7875 AvgReward=0.8724			
Fixed 224 100/100 Acc=0.7900 AvgReward=0.8755			
Fixed budget 224	0.7900	224	0.8755

Evaluating fixed budget = 256...

Fixed 256 40/100 Acc=0.8000 AvgReward=0.8720			
Fixed 256 80/100 Acc=0.8500 AvgReward=0.9345			
Fixed 256 100/100 Acc=0.8300 AvgReward=0.9095			
Fixed budget 256	0.8300	256	0.9095
Policy (DPO deterministic)	0.6700	141.87	0.7666

=====

AI Disclosure: Written with Claude's assistance

[]: